CALLAPICALL • AI VOICE & TELEPHONY SUITE

User Manual

The complete operator & technical reference for the callAPIcall voice, telephony, meeting-copilot and avatar platform on Windows.

Voice sessions (PTT) | Real phone calls | Meeting copilot | Voice cloning | Video avatar | Meeting bot | Android companion | REST API

Edition 1.0 · Covers the call-bridge core, Control Center, avatar worker & companion app

Windows 11 · Python 3.10 · Tauri 2 / React 18

About this manual

This manual is the authoritative operator and technical reference for **callAPIcall**, a suite of AI voice and telephony tools that run on a Windows 11 workstation. It is written for two audiences: **operators** who use the product day-to-day (Parts I and II), and **integrators and developers** who extend it or drive it through its programmatic interfaces (Part III and the appendices).

How to read it. Part I explains what the product is, how its pieces fit together, how to install it and how to configure every setting. Part II is a task-oriented user guide organised around the desktop *Control Center* and the everyday workflows (talking to the agent, placing calls, running the meeting copilot, cloning a voice, driving the avatar). Part III is an exhaustive reference to every software module, function and HTTP endpoint that makes up the system.

Conventions. `Fixed-width` text denotes code, file paths, configuration keys and literal values. Blue call-outs are notes, amber call-outs are warnings, green call-outs are confirmations and purple call-outs are tips. Function signatures are shown exactly as they appear in the source. Keyboard shortcuts are shown as `Shift+F1`. Network ports are given as `:8770`.

Scope & safety. Every default value, parameter name and endpoint path in this manual is taken directly from the shipping source code. The product places and answers real telephone calls, captures system audio and can synthesise a cloned voice and a talking-head avatar; operate it only where you are authorised to record and where all parties consent as required by local law.

Contents

PART I — INTRODUCTION & SETUP

1 · What callAPIcall Is

1.1 Feature overview

1.2 Who it is for

2 · System Architecture

2.1 The processes & ports

2.2 The audio-on-host constraint

3 · Core Concepts

3.1 Half- vs full-duplex & barge-in

3.2 Call transports

3.3 The copilot

3.4 The avatar

3.5 Resilience

4 · Installation & Quick Start

4.1 Prerequisites

4.2 The split architecture

4.3 First test

5 · Configuration Reference

5.1 How configuration is loaded

5.2 Complete key reference

PART II — OPERATOR GUIDE

6 · The Control Center

7 · Voice Sessions & Push-to-Talk

8 · Phone Calls

9 · The Meeting Copilot

10 · Voices, Personas & Studio

11 · Meeting Bot, Calendar & Analysis

12 · The Android Companion App

PART III — TECHNICAL REFERENCE

13 · Command-Line Entry Points

14 · Core Orchestration Modules

15 · Voice & Real-Time Modules

16 · Telephony & Audio Modules

17 · AI Service Clients

18 · REST API Reference

19 · Meeting, Calendar & Analysis Modules

20 · Avatar Worker & Auxiliary Tools

21 · Diagnostics & Probes

APPENDICES

A · Configuration key index

B · Ports & endpoints

C · Glossary

D · Troubleshooting

PART I

Introduction & Setup

What the product does, how its processes fit together, how to install it, and a complete reference to every configuration setting.

CHAPTER 1

What callAPIcall Is

callAPIcall turns a Windows PC into an AI voice and telephony workstation: it talks with you by voice, conducts real cellular phone calls on your behalf, assists you live during video meetings, and can even appear as a talking-head avatar speaking in a cloned voice on a virtual webcam.

At the centre sits **Hermes**, a tool-using AI agent that runs natively on Windows. Around it, the *call-bridge* Python core wires together speech recognition, speech synthesis, telephony hardware, meeting capture and a desktop control application so that a single agent can be reached through many different "front doors" — a hotkey, a phone line, a meeting, or a phone in your pocket.

1.1 Feature overview

Voice conversation

Talk to the agent by voice. Half-duplex on speakers (no echo) or full-duplex on headphones with barge-in.

`Shift+F1 / Shift+F2`

Real phone calls

The agent places and answers real cellular calls, using a USB GSM/4G modem or an Android phone over Bluetooth HFP.

`Calls section / API`

Meeting copilot

Listens to the other party on a call and suggests context + replies in a window hidden from screen-sharing.

`Shift+F4`

Voice cloning

Zero-shot cloning from a 5–15 second sample; the cloned voice is usable everywhere TTS is.

`Voices section`

Video avatar

A lip-synced talking head on a virtual webcam, speaking the agent's replies in the cloned voice.

`Personas / Studio`

Meeting bot & calendar

Auto-joins Zoom/Teams/Meet, transcribes and summarises; can watch your calendar and join on time.

`Meetings section`

Android companion

A phone app that replicates the push-to-talk experience remotely over Tailscale.

`app/`

Control Center

A tray desktop app that drives all of the above and runs five guided self-tests.

`gui/`

1.2 Who it is for

The product is a **single-workstation** system: everything runs on one Windows 11 machine (the audio hardware, the AI backends and the control surface), with only the optional Android companion app running elsewhere. It is designed for a technical individual or a small team who want a private, locally-hosted voice agent with real telephony and meeting integration, rather than a cloud service.

CHAPTER 2

System Architecture

callAPIcall is a set of cooperating processes on one host. Understanding which process owns which port — and why the audio must live on the host — is the key to installing, operating and troubleshooting the system.



2.1 The processes & ports

Process	Port(s)	Role
call-bridge control server (Flask)	`:8770`	REST API consumed by the Control Center and by agents: sessions, personas, config, components, diagnostics, calls, meetings, analysis.
call-bridge voice gateway (WebSocket)	`:8771`	Remote voice endpoint for the Android app (STT → agent → TTS).
call-bridge media server (HTTP)	`:8772`	Serves images/videos generated during a gateway session for the app to download.
Handy (Rust/Tauri)	`:8720`	Speech-to-text (whisper, CUDA) and text-to-speech (Kokoro/Piper/Chatterbox + clones).
Hermes agent	stdio (ACP)	The tool-using AI agent, spoken to over JSON-RPC on standard input/output.
Wikibridge (knowledge)	`:8775`	Optional per-project knowledge/wiki service, started on demand.
Control Center (Tauri)	tray	Desktop control application.

2.2 The audio-on-host constraint

Every audio path — Bluetooth HFP to the phone, microphone/speaker capture, STT and TTS — lives on the Windows host, because that is where the phone's Bluetooth Hands-Free endpoint is reachable. Handy and the native Hermes agent run on Windows too and talk to the core over `localhost` and stdio.

HISTORICAL NOTE

Hermes previously ran inside WSL2, which cannot see Bluetooth audio devices — hence the original "audio on the host" split. After the native cutover the constraint is historical, but the split architecture it produced is still the supported deployment model.

CHAPTER 3

Core Concepts

A handful of ideas recur throughout the product. Understanding them once makes every section of this manual easier to follow.

3.1 Half- vs full-duplex & barge-in

When you talk to the agent by voice there are two duplex modes:

- **Half-duplex** (`Shift+F1` , speakers): while the agent is reasoning and speaking, your microphone is muted, so the speaker output cannot echo back into the microphone. Best when you are using loudspeakers.
- **Full-duplex** (`Shift+F2` , headphones): the microphone is always live, so you can interrupt the agent mid-sentence. This interruption is called **barge-in**. An energy gate (RMS threshold, default `barge_in_energy_threshold = 0.02`) distinguishes real speech from residual TTS echo so that the agent is not interrupted by its own voice.

3.2 Call transports

Real phone calls can travel two ways, chosen by `call_transport` :

- `auto` (default): detect and choose — prefer the USB GSM/4G modem, fall back to the Android phone over Bluetooth HFP.
- `gsm` : force the USB modem (SIM800C on `COM6` by default).
- `android` : force the Android phone (HFP audio).
- `off` : no automatic routing (legacy behaviour).

On the SIM800C/A7670E modems the audio is *turn-based to files* (record a window, synthesise a reply to a file, play it) because those chips do not expose a real-time USB audio stream. See Chapter 16.

3.3 The copilot

The meeting copilot (`Shift+F4`) captures the PC's **loopback** audio — everything the PC is playing, which is the *other party's* voice — and by construction never captures your microphone, so your own speech never triggers a suggestion. It runs STT on that audio, sends it to the agent, and shows a window with a one-line context summary plus `copilot_num_responses` (default 3) suggested replies. Optionally it can speak a chosen reply back (into the call via a virtual audio cable, on your speakers, or through the avatar). The window can be hidden from screen capture (`copilot_hide_from_capture` , default on).

3.4 The avatar

The avatar renders a chosen persona (a photo/clip in `avatar/persone/<name>/`) as a lip-synced talking head on the "OBS Virtual Camera" driver — no OBS Studio required. The **first** activation of a clip pays a one-time preparation cost (frame extraction, DWPose landmarks, VAE latents, blending masks) which is then cached to disk. At runtime an idle loop plays at 25 fps and each reply is pre-rendered (~1–3 s) and played back in sync with the audio.

3.5 Resilience

Two user-session supervisors (registered to autostart under `HKCU`) keep the call-bridge and the Control Center alive, restarting them within seconds of a crash. Stop sentinels (`ptt.stop` , `cc.stop`) mark deliberate stops so the supervisor does not fight an intentional shutdown. Restarting the core from the GUI is implemented as `exit(42)` followed by a supervisor relaunch.

CHAPTER 4

Installation & Quick Start

The supported deployment is the **split architecture**: a headless gateway service plus an interactive push-to-talk process, with Hermes running natively and the Control Center in the tray.

4.1 Prerequisites

Installed once on the Windows 11 host:

Tool	For
Rust (rustup)	Building Handy (STT/TTS engine)
Bun	Handy frontend dependencies
MSVC Build Tools (VS 2022, "Desktop development with C++")	Compiling whisper.cpp etc.
CMake	Required by whisper-rs-sys
Vulkan SDK	Handy GPU acceleration
Python 3.10+	call-bridge, setup, TTS helpers
LLVM / libclang 18.1.1 (via pip)	bindgen for whisper-rs-sys (not LLVM 22)
Android Studio	Building the companion app

LIBCLANG PIN

On this platform LLVM 22 breaks `whisper-rs-sys`. Install an isolated older libclang with `python -m pip install "libclang==18.1.1"` and point `LIBCLANG_PATH` at it for each build shell — do not touch the system LLVM.

4.2 The split architecture

1. **Headless gateway** — an NSSM service `callAPIcall-orchestrator` running `main.py gateway` on `:8771/:8772`. Install with `call-bridge/install-service.ps1`. Runs in Session 0, without hotkeys or overlay.
2. **PTT + control server** in the user session — `main.py all --no-gateway` on `:8770`. Install the task with `call-bridge/install-ptt-task.ps1`; the supervisor `call-bridge/run-ptt.ps1` keeps it alive (auto-restart + heartbeat).
3. **GUI** — `gui/install-autostart.ps1` registers the Control Center to start in the tray; closing the window hides it.
4. **Agent** — Hermes runs natively on Windows (see `cutover-hermes-native.ps1`). With `acp_enabled=true` every brain (F1/F2/F3, gateway, GSM, calls) uses the ACP agent; the old HTTP phone server on `:8780` is no longer needed.

DO NOT USE THE GENERIC INSTALLER FOR PTT

`installer/install.ps1` registers a `main.py serve` task on `:8770` that *conflicts* with the split architecture. Use it only if you understand the conflict. `main.py serve` in a terminal and Hermes-in-WSL are historical/deprecated modes.

4.3 First test

Open the Control Center and run the **Guided Tests** (half-duplex, full-duplex, agent task, phone call, copilot). Each reports its outcome step by step, which is the fastest way to confirm a fresh installation end-to-end.

CHAPTER 5

Configuration Reference

Every runtime setting lives in a single `Config` dataclass. This chapter documents how it is loaded and lists every key with its type, default and effect.

5.1 How configuration is loaded

Configuration is resolved in three layers of increasing priority by `Config.load()`:

1. **Dataclass defaults** — the values baked into `callbridge/config.py` (listed below).
2. **JSON file** — `config.json` beside `main.py` (or an explicit path). Only keys present in the file are overridden.
3. **Environment variables** — `CALLBRIDGE_<KEY>` in upper-case, highest priority. Booleans accept `1/true/yes`; integers and floats are parsed by type; `*_device` keys accept an integer index.

The GUI writes changes back through `Config.save()`, which serialises the whole dataclass to `config.json` with two-space indentation. Helper accessors derive URLs: `base_url()` (phone), `handy_base_url()`, `handy_stt_url()` (`ws://…/stt/stream`) and `handy_tts_url()` (`ws://…/tts/stream`).

> **CHANGE TOKENS BEFORE EXPOSURE**
> The shipped defaults `auth_token`, `control_auth_token` and `gateway_auth_token` are placeholders (`change-me-*`). Replace them before the gateway is reachable from another device.

5.2 Complete key reference

Android control app

Key	Default	Description
`phone_host`	`"192.168.1.50"`	IP of the Android phone on the LAN.
`phone_port`	`8080`	Port of the Android control service.
`auth_token`	`"change-me-shared-token"`	Shared secret; must match `ControlService.AUTH_TOKEN` on Android.

HFP audio (phone's Bluetooth endpoint on Windows)

Key	Default	Description
`hfp_input_device`	`null`	Downlink (other party's voice). `null` = autodetect the target phone's "Hands-Free HF Audio".
`hfp_output_device`	`null`	Uplink (synthesised voice to inject). `null` = autodetect.
`sample_rate`	`16000`	16 kHz (mSBC wideband); falls back to 8 kHz.
`block_ms`	`100`	Audio block size in milliseconds.

Handy STT/TTS engine

Key	Default	Description
`handy_host`	`"127.0.0.1"`	Handy API host.
`handy_port`	`8720`	Handy API port (`HANDY_API_PORT`).
`default_voice`	`"if_sara"`	Default TTS voice.
`default_lang`	`"it"`	Default language.
`default_stt_model`	`null`	`null` = Handy's default STT model.
`tts_speed`	`1.2`	TTS synthesis speed (1.0 = normal, >1 faster).

Control server

Key	Default	Description
`control_host`	`"127.0.0.1"`	Control server bind host.
`control_port`	`8770`	Control server port (GUI & agent API).
`control_auth_token`	`"change-me-control-token"`	Bearer token for the control API.
`poll_status_interval_s`	`1.0`	Phone `/status` poll interval (s).

Push-to-talk

Key	Default	Description
`push_to_talk_enabled`	`true`	Register the global PTT hotkeys.
`push_to_talk_hotkey`	`"shift+f1"`	Full agent, speakers (half-duplex).
`push_to_talk_hotkey_fullduplex`	`"shift+f2"`	Full agent, headphones (full-duplex, barge-in).
`push_to_talk_hotkey_limited`	`"shift+f3"`	Limited model (call simulation).
`ptt_turn_pause_s`	`1.5`	Silence (s) that closes a spoken turn in half-duplex.
`ptt_post_tts_pause_s`	`1.0`	Mic stays muted (s) after TTS ends, to let echo die.
`ptt_listen_warmup_s`	`0.8`	After the pause, mic is live but utterances discarded (s) — STT calibration & anti-false-trigger.
`ptt_input_device`	`null`	PTT input device (`null` = OS default).
`ptt_output_device`	`null`	PTT output device (`null` = OS default).

Copilot

Key	Default	Description
`copilot_enabled`	`true`	Enable the meeting copilot.
`push_to_talk_hotkey_copilot`	`"shift+f4"`	Copilot toggle hotkey.
`copilot_use_acp`	`true`	Use the full ACP agent (tools/search) for the copilot; else HTTP/stub.
`copilot_loopback_samplerate`	`48000`	Sample rate of the loopback output device.
`copilot_lang`	`"it"`	Copilot language.
`copilot_num_responses`	`3`	Number of suggested replies (GUI-adjustable).
`copilot_audio_reply`	`false`	Synthesise the chosen reply and play it.
`copilot_tts_output_device`	`null`	Output device for spoken replies.
`copilot_font_size`	`22`	Initial copilot text size.
`copilot_scroll_speed`	`1.0`	Autoscroll fraction of view/second (0 = off).
`copilot_hide_from_capture`	`true`	Exclude the window from screen capture/recording.
`copilot_preamble`	`(long)`	System preamble instructing the agent to produce CONTEXT + N replies + optional DATA.

Avatar

Key	Default	Description
`avatar_enabled`	`false`	Enable the avatar worker.
`avatar_dir`	`"C:\Projects\callAPIcall\avatar"`	Avatar working directory.
`avatar_persona`	`""`	Persona name under `avatar/persone/` ("" = off).
`avatar_audio_device`	`null`	Device the worker plays the avatar's voice on.
`avatar_gpu_index`	`null`	GPU index for MuseTalk (from `GET /gpu/list`).
`avatar_prepare_timeout_s`	`600.0`	Deprecated — preparation no longer times out.
`avatar_prepare_stall_s`	`300.0`	No-progress watchdog: abort start if the worker stalls this long.

Hermes agent (HTTP & ACP)

Key	Default	Description
hermes_base_url	"http://127.0.0.1:8780"	Base URL for the legacy HTTP phone server.
hermes_mode	"stub"	"stub" = local no-op | "http" = HermesClient.
acp_enabled	true	Use the full ACP agent (tools/actions) for voice and calls.
acp_use_wsl	false	false = native Windows Hermes; true = launch inside WSL.
acp_native_command	hermes.exe acp --accept-hooks	Native launch command (shlex, Windows paths).
acp_cwd_native	"C:\Users\atain"	Working directory for the native session.
acp_hermes_home	%LOCALAPPDATA%\hermes	HERMES_HOME for the native process.
acp_wsl_distro	"Ubuntu"	WSL distro (WSL mode).
acp_wsl_command	(login-shell hermes acp)	Command run in WSL.
acp_cwd	"/home/ataini"	Working directory (WSL mode).
acp_init_timeout_s	240.0	Generous agent init timeout (tools/MCP load).
acp_response_timeout_s	300.0	Max wait for a response (long searches).
acp_voice_preamble	(short)	Minimal preamble: reply in text only, brief and natural.

Barge-in & voice gateway

Key	Default	Description
barge_in_enabled	true	Allow interrupting the agent while TTS plays.
barge_in_energy_threshold	0.02	Minimum RMS (float32) to count audio as speech, not echo.
gateway_enabled	true	Enable the Android voice gateway.
gateway_host	"0.0.0.0"	Bind host (0.0.0.0 for Tailscale reach).
gateway_port	8771	Gateway WebSocket port.
gateway_auth_token	"change-me-control-token"	Shared token with the app.
gateway_media_port	8772	HTTP port serving generated media to the app.

GSM/4G modem

Key	Default	Description
call_transport	"auto"	auto / gsm / android / off.
gsm_port	"COM6"	Modem serial port (CH340).
gsm_baud	115200	Serial baud rate.
gsm_record_window_s	8.0	Per-turn recording window (turn-based, no live VAD).
gsm_answer_wait_s	6.0	Wait for the callee to answer after ATD.
gsm_tts_file	"C:\tts.amr"	Path (on module flash) of the TTS clip to play.
gsm_record_id	1	File id for AT+CREC (1–99; C:\REC\<id>.DVI).
gsm_play_volume	80	CMEDPLAY playback volume (~0–100).
gsm_auto_answer	true	Auto-answer incoming calls.
gsm_max_call_s	600.0	Maximum call duration (anti-loop guard).
gsm_max_empty_turns	3	Consecutive empty turns before hang-up.

Meeting bot, calendar, knowledge & analysis

Key	Default	Description
meetbot_enabled	true	Enable the meeting-bot endpoints.
meetbot_backend	"windows"	"windows" (loopback) | "wsl" (future).
meetbot_display_name	"Hermes Bot"	Name the bot joins meetings under.
meetbot_artifacts_dir	"artifacts/meetings"	Where transcripts/summaries are written.
calendar_google_client_id	""	Google OAuth desktop client id.
calendar_google_client_secret	""	Google OAuth client secret.
calendar_ms_client_id	""	Azure public-client app id (Calendars.Read).
calendar_ms_tenant	"common"	Azure tenant.
calendar_poll_s	300	Calendar poll frequency (s).
knowledge_api_url	"http://127.0.0.1:8775"	Wikibridge knowledge server URL.
knowledge_api_token	""	Knowledge API token.
knowledge_root	"C:\Projects\wikibridge\data"	Knowledge data root.
knowledge_serve_cmd	wikibridge.exe serve	On-demand start command for the server.
analysis_artifacts_dir	"artifacts/analyses"	Analysis artifact folder.
analysis_ffmpeg_path	"ffmpeg"	ffmpeg executable.
analysis_ytdlp_path	"yt-dlp"	yt-dlp executable.
analysis_segment_s	300	Segment length for /transcribe.
analysis_chunk_chars	10000	Map-reduce chunk size (characters).

Component control

Key	Default	Description
handy_exe_path	""	Handy executable for start/stop/restart ("" = disabled).
hermes_start_cmd	""	Command to start Hermes ("" = monitor only).
hermes_stop_cmd	""	Command to stop Hermes ("" = monitor only).

PART II

Operator Guide

Task-oriented instructions for using the product every day, organised around the Control Center and the everyday voice, call, meeting and studio workflows.

CHAPTER 6

The Control Center

Overview

The **Control Center** is a desktop application (Windows) that manages an AI voice and telephony assistant. It runs quietly in the **system tray** and opens to a window with a left **sidebar** of sections and a main content area. The application talks to three local backend services:

- **Control server** ("call-bridge") — the main backend, default `http://127.0.0.1:8770`, secured with a Bearer token read automatically from `call-bridge/config.json`. Drives sessions, calls, personas/avatar, meetings, analysis, config.
- **Handy** — the STT/TTS engine, default `http://127.0.0.1:8720`. Drives voice cloning, TTS engine selection, model load/unload, audio previews.
- **wikibridge** ("Knowledge") — the project knowledge/wiki server, default `http://127.0.0.1:8775`. Started **on demand** by the app the first time the Knowledge section needs it.

A colored dot at the top of the sidebar shows the control server connection: **green "Connected"** when reachable, **red "Disconnected"** when not. Live status is polled automatically — the voice session every 500 ms and the components/health every 5 s (with back-off when the server is down).

The sidebar sections are, in order: **Session**, **Calls**, **Components**, **Guided Tests**, **Voices**, **Identities (Personas)**, **Studio**, **Meetings**, **Knowledge**, **Analysis**, **Handy**, **Settings**. A **language selector** sits at the bottom of the sidebar.

Section: Session

What it is for. This is the home screen. It starts and stops a live **voice conversation** with the assistant (the same thing the global hotkeys Shift+F1/F2 do), shows the current conversation phase, and controls the meeting **Copilot** feature. Use it for a hands-on spoken chat with the agent through your PC's microphone and speakers/headphones.

Controls / fields:

- **Phase pill** — a colored badge showing the live session phase, polled every 500 ms: **idle** (grey), **listening** (blue), **reasoning** (amber, pulsing), **speaking** (violet, pulsing). Reads `GET /session/state`.
- **Mode label** — when a session is active, shows the mode: *Full-duplex*, *Limited*, or *Half-duplex* (from `session.mode`).
- **Half-duplex card** (speaker icon) — starts a session in half-duplex mode: your mic is muted while the agent speaks (best for open speakers). Calls `POST /session/start` with `{mode:"half"}`.
- **Full-duplex card** (headphones icon) — starts a session in full-duplex mode with **barge-in**: you can interrupt the agent while it talks (use headphones to avoid echo). Calls `POST /session/start` with `{mode:"full"}`.
- **Close session** button (red, appears when active) — ends the session via `POST /session/stop`.
- A server-down banner appears if the control server is unreachable; the start cards are disabled. If a session is already running, attempting to start again produces a "session already active" message (HTTP 409).
- **Copilot panel:** Status badge — **active** (green) / **inactive** (`GET /copilot/state`). **Enable/Disable** button — toggles the conversation copilot via `POST /copilot/toggle` (disabled if `copilot_enabled` is off). **Project dropdown** (book icon) — associates the copilot/session with a wikibridge project; sets `POST /copilot/project`. **Save meeting** button — analyzes the current copilot session and saves it as a meeting into the selected project's wiki via `POST /copilot/save-meeting`.

Procedure — hold a voice conversation:

1. Confirm the sidebar dot is green (server reachable).
2. Click **Half-duplex** (speakers) or **Full-duplex** (headphones).
3. Speak; watch the phase pill cycle listening → reasoning → speaking.
4. Click **Close session** when finished.

Procedure — capture a meeting with Copilot:

1. (Optional) Pick a knowledge project in the dropdown.
2. Click **Enable** on the Copilot panel.
3. When the conversation is over, click **Save meeting** to analyze and file it into the project wiki.

Section: Calls

What it is for. Places and ends **real phone calls** in which the AI agent speaks with the person on the line, over whichever transport is available (a GSM modem or a connected Android phone via Bluetooth HFP). Also shows the live phone state.

Controls / fields:

- **Transport detected** line — shows the currently selected transport (`transport.selected`) and its mode (`transport.mode`), from `GET /transport`.
- **Transport mode** dropdown — *Auto* (`""`), *Force GSM* (`gsm`), *Force Android* (`android`). Passed as the `transport` field of the call.
- **Detect** button (refresh) — re-probes transports via `GET /transport?refresh=1`.
- **Number** field — the phone number to dial (required; trimmed).
- **Voice** dropdown — the TTS voice for the call; populated from `GET /models/available` (`tts.voices`), each shown as "voice — language". Empty = default voice.
- **Language** dropdown — *Italian* (`it`), *English (US)* (`en-us`), *French* (`fr-fr`), *Spanish* (`es`).
- **Call** button (green) — dials. Internally it first calls `POST /prepare` (loads TTS, up to 4 min) then `POST /call` with `{number, voice, lang, transport}`. Disabled while busy, unreachable, or already in a call.
- **Hangup** button (red) — ends the call via `POST /hangup`. Enabled only during a call.
- **Phone state** indicator — colored dot + text, refreshed every 2 s from `GET /status` (`phone.state`): **OFFHOOK** (green, in call), **RINGING** (amber), otherwise grey/"Unknown". Shows the number when present.

Procedure — place a call:

1. Click **Detect** to confirm a transport is available (or leave *Auto*).
2. Type the destination **Number**.
3. Optionally choose a **Voice** and **Language**.
4. Click **Call**; the app prepares TTS then dials. Watch the phone-state dot.
5. Click **Hangup** to end.

Section: Components

What it is for. A live health dashboard and start/stop/restart control panel for the backend building blocks: the main bridge, the remote gateway, Handy (STT/TTS), Hermes (the agent), and the avatar worker. Component health is polled every 5 s from `GET /components/status`.

State colors: running (green), loading/stopping (amber), blocked (orange), error/unreachable (red), stopped (grey), disabled (faint).

Cards and their controls:

- **Bridge (main)** — the core control server. Shows `PID` when up. When reachable: **Restart** and **Stop** buttons (`POST /components/main/restart|stop`). When **unreachable**, a single **Start** button appears that performs an offline recovery: it clears the `ptt.stop` sentinel (Tauri `clear_stop_sentinel`), waits up to 8 s for port 8770 to come up on its own, and if still down launches the supervisor script (Tauri `start_supervisor`, runs `run-ptt.ps1` hidden), then waits up to 20 s more.
- **Gateway** — the remote WebSocket gateway (for the companion Android app). Shows state; **Restart bridge** button restarts the main process (the gateway lives inside it).
- **Handy** — STT/TTS engine. **Start / Restart / Stop** buttons (`POST /components/handy/ ...`). Shows a detail line and model status: whether STT and TTS models are **loaded** (with the model name) and the idle-seconds counter.
- **Hermes** — the agent runtime. **Start** and **Restart** buttons (`POST /components/hermes/ ...`).
- **Avatar** — the avatar worker (read-only card, no buttons). Shows the active persona when running.

Procedure — recover a downed backend: If the Bridge card shows **unreachable**, click **Start**; the message area reports "starting supervisor..." / "started" / or "not started" if recovery failed.

Section: Guided Tests

What it is for. Five step-by-step **wizards** that verify the whole pipeline end to end, mixing automatic checks (against the control server's `/diag/*` endpoints and real endpoints) with human confirmations ("did you hear the

reply?"). Results (pass/fail + timestamp) are saved in the browser's localStorage and shown on each test card. There is also an optional **GPU simulation** that estimates how long AI steps would take on a different graphics card.

Top controls:

- **GPU simulation** dropdown — pick a GPU profile to estimate timings, or *No simulation (real GPU)*. Profiles cover dozens of NVIDIA/AMD/Intel/Qualcomm/Microsoft cards and NPUs, each labeled with VRAM · TOPS · bandwidth. During a run, AI-load steps log an estimate like "≈ on RTX 3060: ~8.0s (measured 1.0s, ×8.0)" and flag steps as **not feasible** when the biggest model does not fit the simulated VRAM. (Simulation is purely informational; it never changes what the real machine does.)

The five tests (each opens the wizard: press **Start test**, follow the numbered steps, answer confirmation prompts with "Yes, it works" / "No"; at the first failure the remaining steps are skipped; you can **Stop**/**Re-run**; a running log is shown):

1. **Half-duplex (speakers)** — checks control server, Handy, model load, microphone level (`/diag/mic` , you must speak), starts a half-duplex session, watches the phase cycle listening→reasoning→speaking (ask a question aloud), then asks you to confirm you heard the reply; closes the session.
2. **Full-duplex (headphones)** — same as above in full-duplex, plus a **barge-in** confirmation (interrupt the agent while it speaks). Use headphones.
3. **Agent task (ACP)** — verifies the full tool-using agent: control server, ACP readiness (`/diag/acp` — checks WSL and Hermes agent), model load, starts a session, and asks you to confirm the agent performed a spoken tool task (e.g. "look up tomorrow's weather in Milan").
4. **External phone call** — asks for a test number (your own phone), re-detects the transport, checks Handy and models, dials via `POST /call` , waits up to 45 s for the call to become active (you answer), asks you to confirm you heard the agent, then hangs up.
5. **Video-call copilot** — checks the loopback capture device (`/diag/loopback`), Handy, enables the copilot (Shift+F4 equivalent), asks you to play a talking audio/video and confirm suggestions appeared in the copilot window, optionally reports whether an avatar identity is active, then disables the copilot.

Section: Voices

What it is for. Create and manage **cloned voices** used by the assistant. You record 5–15 s of speech (or upload a clip), give it a name, preview it with your own sample text, set it as the active voice, or delete it. Voice cloning requires a downloaded cloning engine (Chatterbox or ZONOS2); otherwise an amber warning appears. This section talks to Handy directly.

Controls / fields:

- Handy-down red banner if Handy (`:8720`) is unreachable.
- **Cloning-missing** amber warning when no cloning engine is downloaded.
- **TTS engine selector** (shared control) — picks the active Handy TTS engine (Kokoro / Piper / Chatterbox / ZONOS2); only downloaded engines are listed. Sets `POST /tts/model` .
- **New voice — Name** field — letters, digits, `_` , `-` only (validated by `^[a-zA-Z0-9_-]+$`); shows a hint if invalid.
- **Record / Upload** tabs: **Record** — a live recorder. **Start recording** captures mono audio; a live level meter and an elapsed counter "x.x s / 15 s" are shown. Minimum 5 s (Stop is disabled until then), auto-stops at 15 s. Saved as WAV. **Upload** — file picker accepting `.wav, .mp3, .flac, .ogg` . Saving calls `POST /voices/clone` (audio base64-encoded) with the name and format.
- **Sample text** textarea — the phrase used for previews.
- **Cloned voices list** — each row shows the name, a format badge, size, an **Active** badge if selected, an inline **audio player** (Play synthesizes the sample text with that voice via `POST /tts/test`), a **Set active** button (`POST /tts/voice` , switching to a cloning engine if needed), and a **delete** button (`DELETE /voices/cloned/{name}`).

Procedure — clone a voice by recording:

1. Ensure Handy is up and a cloning engine is selected/downloaded.
2. Type a valid **Name**.
3. On the **Record** tab, click **Start recording**, speak 5–15 s, click **Stop**.
4. The voice appears in the list. Edit the **Sample text**, press the row's **Play** to preview.
5. Click **Set active** to make it the assistant's voice.

Section: Identities (Personas)

What it is for. Manage **avatar identities** — named personas, each with a TTS voice and one or more video clips, that drive the talking-head avatar shown through the OBS Virtual Camera. Create/delete personas, assign voices, upload/remove `.mp4` clips, and start/stop the live avatar with a virtual-camera preview.

Controls / fields:

- Server-down red banner when the control server is unreachable.
- **TTS engine selector** (shared).
- **New identity: Name** field (same `^[a-zA-Z0-9_-]+$` rule). **Voice** dropdown — for each cloned voice, two entries "voice (cloned · Chatterbox)" and "voice (cloned · ZONOS2)"; plus stock Kokoro voices `if_sara` (it), `im_nicola` (it), `af_heart` (en), `am_michael` (en); the value encodes `engine:voice`. **Create** button — `POST /personas`.
- **Per-persona card:** header with name, voice badge, **Avatar active** badge; **Start avatar** (`POST /avatar/start` then polls `GET /avatar/state`, no timeout — preparation can take minutes; a progress bar shows frame extraction → face landmark → VAE latents → blend masks, count and ETA); **Stop avatar** (`POST /avatar/stop`); **Delete** (`DELETE /personas/{name}`, disabled while active); inline **Voice** dropdown (`PUT /personas/{name}`); **Clips** — count, **Add clip** (`POST /personas/{name}/clips`), and chips for each clip with an "x" to remove (`DELETE /personas/{name}/clips/{clip}`).
- **Virtual-camera preview** appears when an avatar is running (see Studio).

Procedure — create and launch an identity:

1. Type a valid **Name**, pick a **Voice**, click **Create**.
2. On the new card, click **Add clip** and choose an `.mp4` of the face.
3. Click **Start avatar**; wait for preparation to finish; the preview appears.
4. Click **Stop avatar** when done.

Section: Studio

What it is for. A **TTS + video studio**: choose an identity, type text, and have the avatar **speak it as video** (through the virtual camera), or just preview the **audio only**. Includes a live virtual-camera preview and an interrupt control.

Controls / fields:

- Server-down red banner when unreachable.
- **Identity** dropdown — lists personas ("name — voice"); auto-selects the running avatar's persona. State polled every 4 s.
- **Start avatar / Stop avatar** button — same start-and-wait/stop behavior and preparation progress bar as in Identities. A green "Avatar active: <persona>" note shows when running.
- **TTS engine selector** (shared) — engine used for both the audio preview and the avatar's voice.
- **Text** textarea.
- **Speak** button (clapperboard) — if the avatar for the selected identity is not yet running it first starts it, then synthesizes the video speech via `POST /avatar/speak` (very long timeout). On success shows "spoken" info with voice and duration.
- **Interrupt** button — truncates the current utterance immediately via `POST /avatar/interrupt`.
- **Audio only** — an inline **audio player** that synthesizes the text with the persona's voice via `POST /tts/test` (no video).
- **Virtual-camera preview** — a live video element that attaches to the "OBS Virtual Camera" device and shows exactly what Meet/Zoom/Teams would see. It finds the OBS cam among your webcams, does not steal the camera from a video call, and shows "Avatar off — start it to see the virtual camera" when idle.

Procedure — synthesize a talking-head clip:

1. Pick an **Identity** (must have clips).
2. Optionally choose the **TTS engine**.
3. Type the **Text**.
4. Click **Speak** (auto-starts the avatar if needed) and watch the preview; use **Interrupt** to cut it off.
5. Use the **Audio only** player to preview just the voice.

Section: Meetings

What it is for. A **meeting bot** and meeting archive. It joins a video-call URL (Meet/Zoom/Teams) as a bot, transcribes it live, and afterward produces an AI analysis (summary,

highlights, agreements, tasks) that you can push into a knowledge-project wiki. It also connects **Google/Outlook calendars** to list upcoming meetings and auto-join them.

Join & live bot: Bot status is polled every 3 s (`GET /meetbot/status`). *When idle:* **Title** field, a **project** dropdown, a **URL** field, and a **Join** button (`POST /meetbot/join` with `{url, title, project_id}`). *When active:* a state dot + label with platform, a **Leave** button (`POST /meetbot/leave`), the participant list, and a live transcript tail.

Calendar: Calendar status polled every 30 s (`GET /calendar/status`); events fetched for 7 days (`GET /calendar/events?days=7`). **Google** and **Outlook** buttons — if the provider is **not configured**, clicking opens the **OAuth Setup Wizard**; if configured, clicking runs `POST /calendar/{provider}/connect` (Microsoft returns a **device code** to enter at microsoft.com/devicelogin). A **gear** button opens the wizard to reconfigure. **Event rows** — time, title, provider tag; events with a link show an **Auto-join** checkbox (`POST /calendar/events/{id}/autojoin`) and a **Join now** button.

History & detail: History list — status icon (done/error/analyzing), title, date, participants, segment count; a **delete** button (`DELETE /meetings/{id}`). **Meeting detail** — **Back**; tabs **Summary / Highlights / Agreements / Tasks / Transcript**; a **project** dropdown plus an **Insert into wiki / Republish** button (`POST /meetings/{id}/push-wiki`). The Transcript tab lazy-loads `GET /meetings/{id}/transcript` .

OAuth Setup Wizard (Google / Outlook). A multi-step guided setup opened from the calendar buttons/gears. *Google steps:* create a Google Cloud project; enable the Calendar API; configure the OAuth consent screen; create **Desktop app** OAuth credentials and paste **Client ID** (`calendar_google_client_id`) and **Client secret** (`calendar_google_client_secret`); **Connect** (browser login, read-only calendar). *Outlook/Microsoft steps:* register the app in Azure; add a Mobile/desktop platform with public-client flows; add delegated **Calendars.Read**; paste **Application (client) ID** (`calendar_ms_client_id`) and **Tenant** (`calendar_ms_tenant` , defaults `common`); **Connect** (device code). Each "Open …" link launches the browser via the Tauri `open_url` command; field values are saved via `PUT /config` when advancing.

Procedure — join and file a meeting:

1. Paste the meeting **URL** (optionally a **Title** and a **project**), click **Join**.
2. Watch the live transcript; click **Leave** to exit.
3. Open the meeting from **History**, wait for analysis, review the tabs.
4. Choose a **project** and click **Insert into wiki**.

Section: Knowledge

What it is for. Front-end for the **wikibridge** knowledge server: organize knowledge into **projects**, import external wikis (MediaWiki/Confluence/Notion/generic HTML), ingest local documents, browse and search the generated wiki, and manage auth profiles and import "skill" runtimes. If the server is down, the section shows setup instructions and auto-starts it on demand (Tauri `start_wikibridge`) when an action needs it.

Top controls: Project dropdown (`POST /projects/{slug}/activate`); **New project** field + **Create project** button (`POST /projects`).

Tabs:

- **Import** — **URL** field; **Depth** (0–3); **Scope** (Single page / Subtree / Whole space); **Max pages** (default 200); **Auth** (Public or a saved profile); **Import** button (`POST /imports`); **Import history** with per-item **Sync** (`POST /imports/{id}/sync`); a live progress bar.
- **Documents** — **Upload file** (`POST /ingest`); **Directory path** + **Import directory** (`POST /ingest/dir`); **Drain catalog** (`POST /catalog/drain`).
- **Wiki** — a two-pane browser: a tree grouped into Sources / Entities / Concepts / Syntheses / Meetings (`GET /wiki/tree`) and a page viewer (`GET /wiki/page`); URL-sourced pages show a **Refresh** button (`POST /wiki/refresh`).
- **Search** — full-text search (`GET /search`) with highlighted snippets.
- **Skill / Auth** — **Check status** / **Check and install** the import-skill runtimes (`GET /skill/status` , `POST /skill/install`); **Auth profiles** list + create form (bearer / basic / header / cookie / oauth2; values may reference `env:VAR` ; `POST /auth/profiles`).

Section: Analysis

What it is for. Transcribe and AI-analyze a **video/audio file or URL** (e.g. a YouTube link) into a **summary, highlights, and tasks**, plus a full timestamped transcript.

Controls / fields: Title field (optional); **URL** field + **Analyze URL** button (`POST /analysis/start` with `{url, title}`); **Upload file** button (accepts `.mp4 .mkv .webm .avi .mov .m4v .ts .mp3 .wav .m4a .ogg .flac .aac .opus .wma`; multipart); **History** list (polled every 3 s, `GET /analysis/list`) with delete; **Detail view** — **Back**, source line, a progress bar while active (`GET /analysis/{id}`), tabs **Summary / Highlights / Tasks / Transcript** (transcript from `GET /analysis/{id}/transcript`).

Procedure — analyze a video URL: Paste the **URL**, click **Analyze URL**; the detail opens with a live progress bar; when done, read Summary, Highlights, Tasks, and Transcript.

Section: Handy

What it is for. Direct control of the **Handy STT/TTS engine**: monitor VRAM/model status, load or unload models, and see the current engine and its available voices. Auto-refreshes every 5 s.

Controls / fields: Handy-down red banner if `:8720` is unreachable. **VRAM panel** with **Refresh**; two tiles **STT** and **TTS** (loaded model name or not; TTS shows the selected model); an **idle** line (idle seconds and the auto-unload limit, default 900 s); **Load** (`POST /models/load` with `{tts:true}`) and **Unload** (`POST /models/unload` with `all`). **Voices panel** — active engine, selected model (`GET /models/available`), and chips for each voice.

Section: Settings

What it is for. A full editor for the backend configuration (`GET /config` / `PUT /config`). Fields are auto-grouped by name prefix, each with an inline tooltip and a "needs restart" marker where relevant. Changes are staged and applied together.

Behavior and controls: Fields are grouped into **Session & Voice** (`default_*`, `tts_speed`, `barge_in_*`), **Push-to-talk**, **Copilot**, **Avatar**, **Gateway**, **Phone**, **GSM**, **Hermes** (`hermes_*`, `acp_*`), **Analysis**, **Meeting bot**, **Calendar**, **Knowledge**, **Handy**, **HFP audio**, **Control**, and **Misc**. Each field renders by type: **toggle** for booleans, **number** input for numbers, **textarea** for long fields (`copilot_preamble`, `acp_wsl_command`, `acp_voice_preamble`), a **GPU picker** for any `*_gpu_index` field (from `GET /gpu/list`; "(default)" = null), and text otherwise. A **help (?) icon** shows the tooltip. A floating action bar appears with unsaved changes: **Cancel** and **Save** (`PUT /config`). After saving, if any changed field requires a restart, an amber banner lists them with a **Restart main** button (`POST /components/main/restart`).

Procedure — change a setting: Edit the field(s); note the highlight and the floating bar; click **Save**; if prompted, click **Restart main** to apply restart-only fields.

Language selector

At the bottom of the sidebar, a **language menu** shows the current language by native name; clicking opens a list of 15 languages: English (default/fallback), Italian, Russian, German, French, Spanish, Chinese, Korean, Ukrainian, Portuguese, Hungarian, Hebrew (RTL), Greek, Arabic (RTL), Hindi. The choice persists across sessions. Hebrew and Arabic switch the layout to right-to-left.

The Tauri desktop layer (tray, single-instance, autostart, commands)

System-tray behavior. The app lives in the tray with the tooltip "callAPIcall Control Center" and a menu **Open** / **Quit**: **left-click** the tray icon shows/brings the window to front; **Open** does the same; **Quit** is a *deliberate* exit — it writes a stop sentinel (`gui\cc.stop`) so the GUI supervisor (`run-control-center.ps1`) will not immediately relaunch, then exits. **Closing the window** does **not** quit — it hides the window back to the tray.

Single-instance. Launching a second copy does not open a new window; it shows the existing instance.

Autostart / hidden launch. Passing `--hidden` starts the app directly minimized to the tray.

Invoke commands exposed by the Rust layer (`lib.rs`): `read_bridge_config` (reads host/port/Bearer token from `config.json` at startup); `is_port_open(port)` (TCP

reachability probe); `clear_stop_sentinel` (removes `ptt.stop`); `start_supervisor` (launches `run-ptt.ps1` hidden); `start_wikibridge` (starts wikibridge `:8775` windowless, on demand); `open_url(url)` (opens an http(s) URL in the default browser; rejects non-http(s) URLs).

PART III

Technical Reference

An exhaustive, module-by-module and function-by-function reference to the call-bridge core, its AI service clients, the REST API, and the supporting tools.

CHAPTERS 13–14

Entry Points & Core Orchestration

This chapter documents the command-line entry point and the core orchestration modules of the call-bridge subsystem: the CLI, the orchestration engine, the configuration layer, the HTTP control server, the "brain" factory, shared utilities, and the local Hermes stub. Every default, parameter name, and behaviour is quoted directly from the source.

Module: main.py — CLI entry point

`main.py` is the executable entry point for the call-bridge — the AI telephone-agent host on Windows. It parses a subcommand and dispatches to one of several run modes: audio-device enumeration, connectivity checks, outbound/inbound calls, the headless service, the push-to-talk user session, the voice gateway, or a single combined instance. It wires together the configuration, the orchestrator (`CallBridge`), the various brains, the control server, the voice gateway, and the push-to-talk/overlay stack. STT and TTS are real WebSocket clients to Handy; Hermes is real (`HermesClient`, HTTP/SSE) when `hermes_mode == "http"`, otherwise a local echo stub. When ACP is enabled, the full agent is reached over ACP instead.

Command-line subcommands

Invoked as `python main.py <cmd> [args]`. A subcommand is **required**.

Subcommand	Args	Purpose
`devices`	—	Lists the detected HFP (Bluetooth Hands-Free) audio devices.
`ping`	—	Verifies the connection to the Android control app.
`call`	`number` (required)	Calls a number and starts the conversation (e.g. `+39333xxxxxxx`).
`answer`	—	Waits for one incoming call and answers it (one-shot).
`serve`	—	Starts only the orchestrator control server.
`server`	—	Headless service: control + gateway + media (without PTT).
`ptt`	—	Starts only push-to-talk/overlay (user session).
`gateway`	—	Starts the voice gateway for the Android app (WebSocket) only.
`all`	`--no-gateway` (flag)	Single instance: control server + voice gateway + push-to-talk.

`--no-gateway` (on `all`) skips the voice gateway in this process (it runs in the headless service instead). At startup `main()` calls `_setup_logging()`, then `cfg = Config.load()`, and dispatches on `args.cmd`.

Functions

_setup_logging() -> None

Configures the root logger at `INFO` with a timestamped format.

_cmd_devices() -> None

Resolves HFP input/output indices via `find_hfp_devices()` and prints the downlink (input, interlocutor's voice) and uplink (output, synthesised voice) devices.

_cmd_ping(cfg) -> None

Constructs a `PhoneControlClient(cfg)` and calls `phone.health()`. On success prints OK + `phone.status()`; on failure prints guidance and `sys.exit(1)`.

_cmd_call(cfg, number) -> None

Creates `CallBridge(cfg, StubHermes())`, starts an outgoing call via `start_outgoing_call(number)`, and joins the driver thread. `Ctrl+C` hangs up and joins with a 3 s timeout. This CLI path always uses `StubHermes` (local echo).

_cmd_answer(cfg) -> None

Creates `CallBridge(cfg, StubHermes())`, starts `start_incoming_answer()`, and joins. Same interrupt handling as `_cmd_call`.

_cmd_serve(cfg) -> None

Imports and calls `run_server(cfg)` (blocking) — the plain control-server mode.

_build_brains(cfg, log)

Creates the tuple `(brain_full, brain_limited, brain_copilot)` for push-to-talk. **FULL** (F1/F2) = `make_full_brain(cfg, label="ptt")`. **LIMITED** (F3) reuses the same instance as full when `acp_enabled` (a single hermes process, not two), otherwise `make_limited_brain(cfg)`. **COPILOT** (Shift+F4) is `None` unless `copilot_enabled`; when enabled it is a full brain if `copilot_use_acp`, else limited — a dedicated instance so its context is not mixed with the PTT's.

_start_gateway_thread(cfg, log, critical=False) -> None

Starts the voice gateway in a daemon thread if `gateway_enabled`. When `critical=True` (headless service) a gateway death exits the process with code 42 so NSSM cleanly relaunches; when `False` (`main.py all`, PTT is primary) the failure is only logged.

_build_ptt(cfg, log)

Builds overlay + copilot + PTT manager **without starting anything**, returning `(overlay, manager, copilot)`. Separated from the mainloop so `_cmd_all` can hand the manager/copilot to the control server (`/session/*`, `/copilot/*`) before blocking on the Tkinter loop. It also warms up the ACP agent in a background `acp-warmup` thread so the first F1 does not pay the ~65 s agent-startup cost. The overlay (tkinter) must run on the main thread.

_run_ptt_mainloop(cfg, log, prebuilt=None) -> None

Runs the rich PTT (overlay + F1/F2/F3) on the main thread (blocking). Calls `manager.start()`, prints the hotkey banner, runs `overlay.mainloop()`, and in `finally` always calls `manager.stop()` so global hotkeys are de-registered.

_cmd_ptt(cfg) -> None

Push-to-talk + overlay only (user session). In the split architecture the control server, gateway and media server run in the headless service; this keeps only what needs the interactive session. It does not start a gateway here (that would double-bind on `gateway_port`).

_cmd_all(cfg, no_gateway=False) -> None

Single instance: control server + voice gateway + PTT in one process. Builds PTT first (so GUI endpoints operate on the same hotkey instance), starts the control server in a `control-server` thread with its internal PTT disabled, starts a `models-keepalive` thread (reloads Handy models every 600 s, under the 900 s idle-unload), optionally starts the gateway, then runs the PTT mainloop (blocking). `no_gateway=True` skips the gateway (it runs in the headless service).

_cmd_server(cfg) -> None

Headless service (Session 0): control + gateway (`critical=True`) + media, **without** PTT/overlay. Starts the models keep-alive, then runs `run_server(cfg, start_ptt=False)` (blocking). GUI `/session/*` and `/copilot/*` do not operate here (the PTT manager lives in the user-session process); the hotkeys remain fully functional there.

main() -> None

Sets up logging, builds the argparse parser, parses args, loads config, and dispatches. For `gateway` it calls `run_gateway(cfg)` directly; for `all` it passes `no_gateway=args.no_gateway`.

Operator procedures

- Enumerate audio devices: `python main.py devices`.
- Test the Android app link: `python main.py ping` (exit 1 on failure).
- Place a call (stub brain): `python main.py call +39333xxxxxxx`.
- Answer one incoming call: `python main.py answer`.
- Control server only: `python main.py serve`.

- Headless service (NSSM): `python main.py server` (exits 42 if the gateway dies).
- User-session PTT only (split): `python main.py ptt`.
- Voice gateway only: `python main.py gateway`.
- Everything in one process: `python main.py all` (add `--no-gateway` when the gateway runs in the service).

Module: config.py

Defines the `Config` dataclass — the single source of truth for all call-bridge settings, with code defaults, JSON-file overrides (`config.json`) and `CALLBRIDGE_*` environment-variable overrides (highest priority). **The complete list of every configuration key, its type, default and effect is given in Chapter 5 (Configuration Reference); it is not repeated here.** The methods below are the loader and helper accessors.

@classmethod load(cls, path=None) -> Config

Populates a `Config`: (1) code defaults; (2) if the JSON file exists, overrides any field present in it; (3) `CALLBRIDGE_<FIELD>` environment variables override everything. Env coercion: booleans accept `1` / `true` / `yes`; integers via `int()` (also for `None`-valued `*device` fields); floats via `float()`; else string. When `path` is `None`, uses `config.json` next to `main.py`.

base_url() -> str

Returns `http://<phone_host>:<phone_port>`.

handy_base_url() -> str

Returns `http://<handy_host>:<handy_port>`.

handy_stt_url() -> str

Returns `ws://<handy_host>:<handy_port>/stt/stream`.

handy_tts_url() -> str

Returns `ws://<handy_host>:<handy_port>/tts/stream`.

as_dict() -> dict

Returns the full config as a dictionary (`dataclasses.asdict`).

save(path=None) -> None

Writes the current configuration to a JSON file (`indent=2`, UTF-8). When `path` is `None`, writes to `config.json` next to `main.py`.

Module: orchestrator.py

The call-lifecycle orchestrator. `CallSession` represents an active phone call (a `StreamingSession` plus a watcher that closes the session when the call returns to IDLE).

`CallBridge` manages the whole lifecycle — preparing models, placing/answering calls, running the streaming session — and is the entry point used by both the CLI and the control server.

class CallSession(StreamingSession)

__init__(self, config, hermes, in_idx, out_idx, hfp_rate, voice, lang, phone) -> None

A single phone call's session. `config` (Config); `hermes` (the brain, duck-typed `respond(text) → Iterator[str]`, or a `HermesSession`); `in_idx` / `out_idx` (HFP device indices); `hfp_rate` (negotiated rate); `voice` / `lang` (TTS); `phone` (a `PhoneControlClient`).

_extra_threads(self) -> list

Returns one daemon `state-watcher` thread targeting `_state_watcher`.

_state_watcher(self) -> None

Every `poll_status_interval_s`, if `phone.status()["state"] == "IDLE"` it logs, sets `_stop`, and returns (the far end hung up). `PhoneError` is swallowed and retried.

class CallBridge

__init__(self, config, hermes) -> None

The orchestrator: call lifecycle + streaming sessions.

Creates a `PhoneControlClient`, a `HandyControlClient`, and a session lock. `hermes` is a `HermesClient` or a stub with `.respond(text)`.

prepare(self, stt_model=None, tts=True) -> dict

Loads models into VRAM ahead of a call via `handy.load_models( ... )`; returns Handy's result.

start_outgoing_call(self, number, voice=None, lang=None, stt_model=None) -> threading.Thread

Starts an outgoing call (non-blocking); returns the `call-driver` thread. `None` voice/lang/stt_model fall back to the config defaults.

start_incoming_answer(self, ring_timeout=30.0) -> threading.Thread

Waits for one incoming call, handles it, then terminates (one-shot). Returns the `answer-driver` thread.

start_incoming_watcher(self) -> threading.Thread

Continuously watches for and handles every incoming call (the `incoming-watcher` thread).

hangup(self) -> dict

Stops the active session (if any), then calls `phone.hangup()`. Returns `{"status":"ok"}` or `{"status":"error","message": ... }`.

status(self) -> dict

Returns `{"session_active", "phone", "handy"}` (phone/handy errors embedded as `error` fields).

Internal drivers. `_drive_outgoing_call` (load models → resolve devices → dial → wait OFFHOOK → run session); `_handle_incoming_call` (read caller, derive lang/voice via `voice_lang_for_number`, load TTS, resolve devices, answer, run session); `_wait_for_incoming` (poll until RINGING+INCOMING); `_run_session` (atomically reserve the session slot, wrap a `HermesClient` in a `HermesSession` with `notify_*` / `call_ended`, run the `CallSession`, safe-hangup on error); `_resolve_devices` (autodetect HFP indices + negotiate rate); `_wait_for_state`; `_safe_hangup`.

Module: control_server.py

The orchestrator's HTTP control server (Flask). The orchestrator is the **server** for commands from Hermes/the skill and for outgoing calls; it also mounts the Control Center GUI blueprints and drives the transport router. Auth is `Authorization: Bearer <control_auth_token>` on every route except `/health`.

Endpoints (built in create_app)

Method & path	Purpose
`GET /health`	Liveness (no auth). `{"status":"ok"}`.
`GET /status`	Orchestrator + phone + Handy models status.
`POST /prepare`	Load models. Body `stt_model?`, `tts?` (default True).
`POST /call`	Outgoing call. Body `number` (req), `voice?`, `lang?`, `greeting?`, `transport?`, `stt_model?`.
`POST /hangup`	Terminate the current call (via router if present).
`GET /models/available`	Proxy → Handy voices/languages.
`POST /models/load`	Proxy → Handy load. Body `stt_model?`, `tts?`.
`POST /models/unload`	Proxy → Handy unload. Body `component?` (default `all`).
`GET /models/status`	Proxy → Handy VRAM status.
`GET /transport`	Detection + selected transport. `?refresh=1` re-detects.

The GUI blueprints `api_session`, `api_personas`, `api_config`, `api_components`, `api_diag`, `api_gpu`, `api_analysis`, `api_meetings` are also registered (documented in Chapter 18).

create_app(config, bridge=None, ptt=None, copilot=None) -> Flask

Builds the app. If `bridge` is omitted, one is created with `StubHermes`. `ptt` / `copilot` are the shared instances for `/session/*` and `/copilot/*` (absent ⇒ those endpoints answer 503). Sets a 2 GiB upload cap (persona clips /

analysis media), stores shared references in `app.config`, and — if `call_transport` is not `off` — builds a lock-guarded `CallRouter` and calls `setup()`. A `before_request` gate enforces the Bearer token (except `/health`); an `after_request` hook adds permissive CORS for the Tauri webview. `/call` normalises and validates the number, then delegates to the router or the legacy bridge.

run_server(config, bridge=None, start_ptt=True, ptt=None, copilot=None) -> None

Starts the control server, the incoming-call watcher, and (optionally) push-to-talk (blocking). If `bridge` is omitted, one is created with the config-selected brain (ACP native if `acp_enabled`, else `HermesClient` if `hermes_mode=="http"`, else `StubHermes`). With `start_ptt=False` the rich PTT is managed elsewhere in the same process. Finally runs `app.run(host=control_host, port=control_port, threaded=True)`.

Module: brains.py

Factory for the conversational brains plus a unified `respond()` dispatch, centralising the back-end selection cascade (full ACP → Hermes HTTP → stub).

make_full_brain(cfg, label="")

The FULL brain (F1/F2, gateway, GSM). Returns `AcpAgentClient(cfg)` if `acp_enabled`; else `HermesClient(cfg)` if `hermes_mode=="http"`; else `StubHermes()`. Logs the selection.

make_limited_brain(cfg)

The LIMITED brain (F3, call simulation) — same cascade. After the native cutover, callers should share the full brain instead of building a second one, to avoid a second hermes process.

respond_stream(brain, text, cfg, *, preamble=None, session_tag="app", lang=None) -> Iterator

Queries any brain and yields chunks, smoothing signature differences. For `AcpAgentClient`: `brain.respond(text, preamble=preamble)`. For `HermesClient`: creates a session id `<tag>-<12hex>`, notifies outgoing, wraps in a `HermesSession`, yields, and ends in `finally`. For a stub: `brain.respond((preamble or "") + text)`.

Module: util.py

Shared helpers for the voice pipeline plus safety validators.

is_safe_artifact_id(value) -> bool

`True` if `value` matches `^[a-zA-Z0-9_-]+$` — no path separators or dots (path-traversal defence for artifact ids).

normalize_phone_number(value) -> str

Removes decorative separators (`[\s\-().]`).

is_valid_phone_number(value) -> bool

`True` if (already normalised) it matches `^\+?[0-9*#]{2,20}$` — AT-command-injection defence for the `ATD<number>;` line.

drain_queue(q) -> int

Empties a queue without blocking; returns how many items were discarded.

coalesce_utterances(q, first, pause_s, stop=None) -> str

Merges closely-spaced STT utterances into one turn: keeps collecting until a silence pause of `pause_s`, joining fragments with a space. `stop` (an Event) aborts early.

Module: __init__.py

Package marker. `__version__ = "0.1.0"`.

Module: stubs.py

A stub Hermes that lets the orchestrator run end-to-end before the real agent integration. STT and TTS are **not** stubs — they are the real Handy WebSocket clients.

class StubHermes — respond(self, text) -> Iterator[str]

Builds `reply = f"Ho sentito: {text}. "` and yields it word-by-word (simulating token streaming).

Cross-module notes. The restart exit code 42 is shared between `main._start_gateway_thread` and the GUI restart command (`api_components`). The models keep-alive loop (600 s) is intentionally shorter than Handy's 900 s idle-unload. ACP brains are shared (not duplicated) between the full (F1/F2) and limited (F3) paths when `acp_enabled` is true.

CHAPTER 15

Voice & Real-Time Modules

This chapter covers the real-time voice interaction subsystem: push-to-talk hotkey dialogue, the on-screen brain overlay, the meeting/conversation copilot, the shared audio streaming engine, the remote WebSocket voice gateway, and the telephony transport router.

Module: ptt.py — Push-to-Talk

Push-to-talk lets a user talk directly to the assistant using a global keyboard hotkey. Pressing the hotkey opens a spoken session on the operating system's **default** audio devices (the PC's mic and speakers) — distinct from the Bluetooth HFP endpoint used for actual phone calls. Pressing the same hotkey again closes the session. The module owns three PTT hotkeys (full agent half-duplex, full agent full-duplex, limited/simulation) plus an optional fourth that toggles the copilot.

class PushToTalkSession(StreamingSession)

A thin subclass of `StreamingSession` that runs on the OS default audio devices with no call-state watcher (there is no phone call to monitor).

PushToTalkManager.__init__(self, config, brain_full, brain_limited=None, overlay=None, copilot=None)

Constructs the manager that listens for hotkeys and opens/closes sessions. `brain_full` is the COMPLETE agent (F1/F2); `brain_limited` is the LIMITED model (F3; falls back to `brain_full` if `None`); `overlay` is the `BrainOverlay` (or `None`); `copilot` is the `CopilotController` (enables Shift+F4 when present). Internal state: `_session`, `_starting` (guards concurrent starts), `_lock`, `_listener`, `_mode`.

PushToTalkManager.start(self) -> None

Registers the global hotkeys (non-blocking). If `push_to_talk_enabled` is false it returns without registering (avoids hotkey-ownership conflicts). Binds F1 (full, `half_duplex=True`, `mode="half"`), F2 (full, `half_duplex=False`, `mode="full"`), F3 (limited, `half_duplex=True`, `mode="limited"`), and — if a copilot exists — the copilot hotkey to `_toggle_copilot`. Starts a `keyboard.GlobalHotKeys` listener thread.

PushToTalkManager.stop(self) -> None

Stops the hotkey listener, stops the active session, and deactivates the copilot if present.

PushToTalkManager.start_session(self, mode) -> tuple[bool, str]

Programmatic session start for the control server/GUI. **Idempotent** (never toggles off): if a session is active it is NOT closed. `mode` ∈ `"half"` (≡F1), `"full"` (≡F2), `"limited"` (≡F3); any other value returns `(False, "modalita' sconosciuta: <mode>")`. Returns `(True, mode)` on success, `(False, "session_active")` / `(False, "starting")` if busy, or `(False, "start_failed")` if models aren't ready or audio devices are unavailable.

PushToTalkManager.stop_session(self) -> bool

Closes the active session if present; returns whether there was one.

PushToTalkManager.session_state(self) -> dict

Returns GUI state: `{"active", "mode", "phase"}` where `phase` mirrors the overlay logic (speaking > reasoning > listening), or the idle triple when no session.

PushToTalkManager._toggle(self, brain, half_duplex, mode) -> None

Internal hotkey handler: closes an existing session, or reserves `_starting` and starts a new one outside the lock (so the slow preload doesn't freeze other hotkeys).

PushToTalkManager._start_session_reserved(self, brain, half_duplex, mode=None) -> bool

Prepares and starts the session (must be called with `_starting=True` and without `_lock`). Reads `ptt_input_device` / `ptt_output_device`; calls `ensure_models_loaded` (returns `False` if models aren't loaded — the session opens `/tts/stream` immediately and would time out); negotiates the sample rate; if `brain` is a `HermesClient` wraps it in a `HermesSession` (`ptt-<12hex>`, `notify_outgoing`); constructs a `PushToTalkSession`; wires the overlay state function; and starts the `ptt-session` thread.

PushToTalkManager._stop_session_locked(self) -> None

Logs, calls `session.stop()`, clears state, resets the overlay to hidden, and calls the saved `_session_end` (notifies Hermes "call-ended") if set.

PushToTalkManager._to_pynput(hotkey) -> str (staticmethod)

Converts a config hotkey like `"shift+f1"` into pynput's `"<shift>+<f1>"`. Recognised specials (wrapped in `<...>`): `ctrl, alt, shift, cmd, space, tab, esc, enter, up, down, left, right`, and `f1`..`f24`.

Module: ptt_overlay.py — Brain Overlay

A small always-on-top window with a brain icon (🧠) that mirrors the PTT session state. It must run on the **main thread** (tkinter is not thread-safe). Other threads only change state by assigning a callable via `set_state_fn()`. If tkinter is unavailable the overlay becomes a no-op. Colours: `#2563eb` (listening), `#d97706` (reasoning), `#7c3aed` (speaking). Window size 150×96. States map to (icon, colour, label): listening→(🧠, blue, "In ascolto"), reasoning→(💭, amber, "Elaboro..."), speaking→(🗣️, purple, "Hermes parla").

BrainOverlay.set_state_fn(self, fn) -> None

Sets the state source (a callable returning one of `hidden` / `listening` / `reasoning` / `speaking`). Callable from any thread.

BrainOverlay.post(self, fn) -> None

Runs `fn` on the tk thread (thread-safe); no-op if tk is unavailable.

BrainOverlay.mainloop(self) -> None

Main-thread entry: creates a borderless, always-on-top, initially invisible window centred near the bottom of the screen, schedules `_tick`, and enters the tk mainloop. On a `TclError` (e.g. Session 0) it falls back to `_block_no_ui()` so the PTT process keeps running (hotkeys still work).

BrainOverlay._tick(self) -> None

Every 80 ms: drains the command queue, reads the state function, and shows/hides the window. Listening is a fixed opacity 0.93; reasoning/speaking pulse via a sine of the frame counter.

Module: copilot.py — Conversation Copilot (Shift+F4)

The copilot opens a window that listens to the OTHER party of a call/meeting playing on the PC and suggests a brief context plus 2–3 candidate replies. The user's own voice is **never** captured: the mechanism is **loopback capture** — it records the PC's audio OUTPUT (what comes out of the speakers = the other party's voice) via the `soundcard` library (WASAPI loopback). No mic, no ML, no calibration. Threads: `copilot-capture` (loopback → resample → Handy STT), `copilot-brain` (STT → coalesce → brain → UI queue), and the tk thread.

@dataclass CopilotEntry

One suggestion block: `seq`, `utterance` (header), `body` (accumulated response), `done`.

CopilotSession.__init__(self, cfg, brain, ui_post, window, avatar=None, project_id=None)

The worker pipeline. `ui_post` posts UI callbacks to the tk thread; `avatar` (an `AvatarClient` or `None`) routes "reply with audio" through the video worker when active; `project_id` links a wikibridge project. Tracks a `transcript` for "save meeting to wiki".

CopilotSession.start(self) -> None

Creates and starts a `HandySttClient`; launches `copilot-capture` and `copilot-brain`. If an avatar + persona is configured, warms the avatar in a background thread (text copilot works meanwhile).

CopilotSession.speak(self, text) -> None

Synthesises `text` (Handy TTS) and plays it on the chosen device. To speak INTO the call, set `copilot_tts_output_device` to a virtual audio cable used as the call app's microphone.

CopilotSession._capture(self) -> None

Loopback capture thread. Opens `sc.get_microphone(str(spk.name), include_loopback=True)` as a mono recorder at `copilot_loopback_samplerate`, records blocks, resamples to 16 kHz, and sends to STT; retries after 1 s on device error.

CopilotSession._brain_loop(self) -> None

Coalesces STT fragments into one turn (`coalesce_utterances`, `ptt_turn_pause_s`), builds a `CopilotEntry`, records a transcript entry (speaker "Interlocutore"), posts it to the window, then streams `_copilot_respond( ... )` appending chunks and marking `done` at the end.

CopilotWindow (tk window, on the tk thread)

A large scrollable text window (720×900, top-right, always-on-top) with adjustable autoscroll and font size. Bottom bar: `A−` / `A+` (font), `« lento` / `» veloce` (speed), `⏯ pausa`, and a capture-visibility toggle. Second bar: a "Risposte" counter (1–6) and a "🔊 Rispondi con audio" checkbox. Color tags: `hdr`, `ctx`, `reply` (click-to-read), `data`, `note`. `_apply_capture_affinity` calls `SetWindowDisplayAffinity` with `WDA_EXCLUDEFROMCAPTURE (0x11)` when hidden, excluding the window from screen sharing/recording (Win10 2004+).

CopilotController.__init__(self, cfg, brain, overlay)

Creates/destroys the session + window on each Shift+F4 toggle. Holds `save_state = {"state":"idle"}` and keeps `_last_session` so a closed session can still be saved.

CopilotController.save_meeting_async(self, project_id=None, title=None) -> tuple[bool, str]

Analyses the session transcript (active or last) and inserts it into the project wiki. Asynchronous — `save_state` advances `idle` → `saving` → `done` / `error`. Guards: no transcript, no project, or already saving each return an error. On success stores the `wiki_path`.

CopilotController.toggle(self) -> None

`_activate()` if no session, else `_deactivate()`. `_activate` builds an `AvatarClient` when `avatar_enabled` and a persona is set.

Module: streaming.py — Audio Streaming Engine

The shared bidirectional session used by both phone calls (`CallSession`) and push-to-talk (`PushToTalkSession`). Three worker threads — downlink (audio in → 16 kHz → Handy STT), brain (STT utterance → Hermes → Handy TTS), uplink (Handy TTS → audio out) — implement half- vs full-duplex logic and the barge-in/energy-gate heuristic.

StreamingSession.__init__(self, config, hermes, in_idx, out_idx, audio_rate, voice, lang, half_duplex=False)

`hermes` is a duck-typed brain (`respond(text) → Iterator[str]`). `half_duplex=True` forwards no audio to STT while TTS plays (prevents the mic re-capturing TTS from speakers) — used by PTT with speakers; phone calls stay full-duplex. Internal Events: `_stop`, `_tts_active`, `_processing`, `_barge_in`, `_listening`, `_cooldown`.

StreamingSession.phase (property) -> str

Derives the phase from the internal Events: `speaking` if `_tts_active`; else `reasoning` if `_processing`; else `listening`.

StreamingSession._looks_like_echo(self, utterance) -> bool

`True` if the utterance is TTS echo of what was just spoken. Conservative: `False` under 3 words / no last response; `True` if the words appear verbatim in the last response, or (≥6 words) if the word-overlap ≥ 0.85.

StreamingSession.run(self) -> None

Opens and starts `HandySttClient` and `HandyTtsClient`, configures TTS (`voice`, `lang`, `speed=tts_speed`), opens `HfpAudioIO`, starts the three worker threads plus `_extra_threads()`, waits on `_stop`, joins, closes.

StreamingSession._downlink(self, audio, stt) -> None

Audio in → resample 16 kHz → STT. Half-duplex: skip while `_processing` / `_cooldown`. Full-duplex: while TTS plays, only above-threshold audio (`barge_in_energy_threshold`) passes (filters residual echo — a heuristic gate, not a true AEC).

StreamingSession._brain(self, stt, tts) -> None

STT utterance → Hermes (streaming) → Handy TTS, with barge-in. Half-duplex coalesces fragments into one turn and discards echo. The reply is fully accumulated (cleaned into sentences via `TtsTextFilter`) and sent to TTS in ONE shot at end-of-generation for one continuous audio stream. Full-duplex abandons the reply if a new utterance is queued (barge-in). A `dict` chunk carrying `voice_change` switches TTS voice/lang at runtime. In `finally` (half-duplex) it clears `_processing`, sets `_cooldown`, waits `ptt_post_tts_pause_s + ptt_listen_warmup_s`, drains STT, and resumes listening.

StreamingSession._uplink(self, audio, tts) -> None

Handy TTS → resample → audio out. Skips segments whose `generation ≠ tts.generation` (leftovers from an aborted reply), sets `_tts_active`, and writes in device-block-sized chunks, breaking early on `_stop` / `_barge_in`.

StreamingSession._interrupt_tts(self, tts) -> None

Aborts the current TTS: sets `_barge_in`, calls `tts.bump_generation()` (old-generation segments are discarded by `_uplink`), and drains the audio queue.

Module: voice_gateway.py — Remote WebSocket Voice Gateway

A WebSocket server bridging a remote app (over Tailscale/LAN) to Handy/Hermes — replicating Shift+F1 but with audio flowing to/from the app. It processes **one turn at a time** for viva-voce modes (hold/continuous), and a **full-duplex** streaming session for earpiece/Bluetooth. The protocol mixes JSON text messages with binary PCM16 frames (int16 little-endian mono at the negotiated sample rate).

The WebSocket protocol

app → gateway: `{"type":"auth","token":"…"}`; `{"type":"start","mode":"hold"|"continuous","lang","voice","sample_rate"}` (plus optional `"duplex":true`); binary audio frames; `{"type":"end"}` (hold only); `{"type":"image","data","mime"}`; `{"type":"stop"}` / `{"type":"close"}`.

gateway → app: `{"type":"auth_ok"}` / `ready` / `session_started`; `{"type":"transcript","text"}`; `{"type":"reply","chunk"}`; `{"type":"audio_start","sample_rate"}`, binary reply-audio frames, `{"type":"audio_end"}`; `{"type":"media","kind","url"|"token","port"}`; `turn_done` / `interrupt` / `done`; `{"type":"error","message"}`.

The WS server listens on `gateway_host:gateway_port`; a media HTTP server listens on `gateway_media_port` (8771 / 8772 in this deployment).

_pcm16_to_f32 / _f32_to_pcm16

Convert between int16-LE bytes and float32 in `[-1,1]`.

_normalize_audio(samples) -> np.ndarray

Removes DC offset and attenuates ONLY if the signal clips (never amplifies — amplifying near-silence would turn background noise into "speech" and cause hallucinated transcripts).

_ReplyFilter

Extracts Hermes media tags `[[media:image:SRC]]` / `[[media:video:SRC]]` from the reply stream and returns the clean display text, holding back a partial trailing tag.

_Connection (one app connection, multiple turns)

`_do_turn` (viva-voce, one turn: configure TTS, capture the full utterance, send transcript, stream the reply, forward TTS audio, `done`); `_do_duplex_session` (full-duplex: a `gw-duplex-rx` thread feeds app audio to STT, the main loop streams replies with a barge-in callback); `_capture_utterance` (returns the COMPLETE transcribed utterance up to a credible pause; `hold` waits for `{end}`, `continuous` closes on the natural pause); `_forward_tts_audio` (one `audio_start`, streamed segments via `_send_pcm` in ~8 KB sub-frames, one `audio_end`); `_stream_reply` (app gets RAW text for markdown; TTS gets SANITIZED text via `TtsTextFilter`); `_authenticate` (token check against `gateway_auth_token`).

run_gateway(cfg) -> None

Starts the media HTTP server, builds the full brain (`make_full_brain(cfg, label="gateway")`), and serves connections (each handled by `_Connection(ws, cfg, brain).run()`) on `gateway_host:gateway_port`.

Module: transport.py — Telephony Transport Router

Automatic transport selection for phone calls: **gsm** (a USB SIM800C modem over serial, preferred) and **android** (an Android phone via Bluetooth-HFP + control app, fallback). "Available" isn't enough — the transport must be able to **place a call** (GSM = registered with SIM; Android = control app reachable). Policy via `call_transport`: `auto` (default) | `gsm` | `android` | `off`.

@dataclass TransportInfo

`name`, `present`, `can_call`, `detail`.

_probe_modem(dev, baud) -> bool

Opens the port and returns `True` if it answers `OK` to `AT`.

find_gsm_port(preferred, baud) -> str | None

Searches for the modem port (preferred name, then CH340 adapters VID `0x1A86` /PID `0x7523`, then other non-Bluetooth serial ports) and returns the first that answers `AT`.

detect_gsm(cfg, existing=None) -> TransportInfo

Detects the modem (reusing an already-open `GsmModem` if given); `can_call = status["registered"]`.

detect_android(cfg) -> TransportInfo

`present` / `can_call` both equal the phone-control-app `health()` result.

select(cfg, existing_gsm=None) -> tuple[str | None, dict]

Detects and chooses: `gsm` → gsm if it can call; `android` → android if it can call; `auto` → gsm else android else `None`; `off` → `(None, {})`.

CallRouter

Routes calls onto the auto-selected transport. `setup()` (detect + open the modem if GSM is preferred); `refresh()` (re-detect, skipped while a GSM call is active); `_gsm_incoming` (auto-answer via a `GsmCallSession` when `gsm_auto_answer`); `place_call(number, voice=None, lang=None, greeting=None, force=None)` (GSM → `run_outgoing` in a `gsm-call` thread; Android → `bridge.start_outgoing_call`; returns `{"status":"calling","transport", ...}`); `hangup()` (hangs up both transports); `status()`.

Key concepts (for users)

- **Half- vs full-duplex.** Half-duplex (F1/speakers, F3/limited) mutes the mic while the assistant thinks and speaks (plus a short post-TTS cooldown), stopping the mic from re-hearing the assistant. Full-duplex (F2/headphones) keeps the mic open, so you can barge in.
- **Barge-in / energy gate.** In full-duplex, while TTS plays only audio above `barge_in_energy_threshold` reaches the STT (rejects residual echo — not a true AEC). A real new utterance aborts the current reply and is processed immediately. Enabled by `barge_in_enabled`.
- **The four hotkeys.** F1 = full agent, half-duplex, speakers; F2 = full agent, full-duplex, headphones; F3 = limited call-simulation model; Shift+F4 = copilot toggle.
- **Copilot loopback capture.** The copilot records the PC's OUTPUT (WASAPI loopback), which is the other party's voice only — your microphone is never touched, so your voice is excluded by construction.

CHAPTER 16

Telephony & Audio Modules

This chapter covers the physical/transport layer that connects the assistant to real phone calls. There are two independent paths: **Android + Bluetooth HFP** (the phone dials on the cellular network and routes call audio to the PC as an audio device, controlled by an HTTP `PhoneControlClient`), and a **direct GSM/4G USB modem** (a SIM-equipped USB modem dials directly; on the SIM800C the audio is exchanged turn-based via files on the modem's flash).

Hardware concepts

Transport auto-selection

`call_transport = auto` prefers the USB GSM/4G modem and falls back to the Android phone over HFP. The two paths are complementary: the Android HFP path uses `PhoneControlClient` (HTTP to the phone app) for control and `HfpAudioIO` for real-time, full-duplex, low-latency audio; the direct GSM path uses `GsmModem` (AT commands) for control and file-based, turn-based audio (`CREC` / `CMEDPLAY`), one fixed-window turn at a time with no downlink VAD.

Why real-time USB audio is unsupported on the SIM800C

The SIM800C only exposes call audio on analog pins that are not broken out on this board — there is no USB audio interface. So voice moves **to file, turn-based**: play a file into the call with `AT+CMEDPLAY`, record the call to a file with `AT+CREC`, and transfer files over serial with the flash file system (`FSCREATE/FSWRITE/FSREAD/FSFLSIZE/FSDEL`). The A7670E 4G dongle on COM9 likewise has no usable USB audio stream and uses a different AT dialect. True real-time USB call audio would need a module that presents a USB sound card (e.g. Quectel EC25 / SIMCom SIM7600).

Module: audio_hfp.py

Real-time audio I/O over the Bluetooth HFP endpoint that the Android phone exposes to Windows. **Downlink** (device input) is the remote caller's voice, fed to STT; **uplink** (device output) is the synthesised TTS voice sent into the call. The Bluetooth SCO link only activates when the streams open, so the HFP device is silent until a call is routed to the PC. Constants: `PHONE_NAME_HINTS = ("s25","andrea","ultra")`, `HFP_NAME_HINTS = ("hands-free","headset","hands free")`, `EXCLUDE_HINTS = ("a2dp","stereo")` (A2DP is stereo playback with no microphone).

find_hfp_devices() -> tuple[int, int]

Auto-detects the phone's HFP endpoint and returns `(input_idx, output_idx)`. Prefers an input/output pair that share the same device name (avoids reading one device but writing another). Scans first with `PHONE_NAME_HINTS`, falling back to `HFP_NAME_HINTS` with a warning. Raises `RuntimeError` if no valid pair is found (advising to set `hfp_input_device` / `hfp_output_device` manually).

negotiate_sample_rate(in_idx, out_idx, preferred) -> int

Returns `preferred` if both input and output streams accept it (mono), else `8000` (narrowband fallback), logging a warning.

resample(samples, from_rate, to_rate) -> np.ndarray

Resamples mono float32 audio via `soxr`; returns the input unchanged if the rates match.

class HfpAudioIO

A context manager opening the HFP input/output streams. `__init__(in_idx, out_idx, sample_rate, block_ms)` computes `block_frames = int(sample_rate*block_ms/1000)`. `__enter__` opens and starts mono float32 input/output streams; `__exit__` stops

and closes them. `read()` returns one downlink block (float32 mono); `write(samples)` reshapes to `(-1,1)` float32 and writes one uplink block.

Module: gsm_audio.py

Audio codec bridge for the file-based GSM path. The SIM800C records/plays AMR-NB (8 kHz), while `CREC` recordings are DVI/IMA ADPCM; the internal pipeline uses PCM float32 mono (16 kHz for STT, 24 kHz for TTS). Constant `FFMPEG = "ffmpeg"` (must be on PATH).

pcm_to_amr(samples, sample_rate, bitrate="12.2k") -> bytes

Encodes PCM float32 mono to an in-memory AMR-NB (8 kHz) file via ffmpeg (`libopencore_amrnb`). Used for the TTS-to-caller direction (uploaded and played with `CMEDPLAY`). `12.2k` is the highest AMR-NB mode. Raises `RuntimeError` on timeout/non-zero exit.

amr_to_pcm(amr, out_rate=16000) -> np.ndarray

Decodes an AMR-NB file to PCM float32 mono at `out_rate` (16 kHz for STT).

dvi_to_pcm(dvi, in_rate=8000, out_rate=16000) -> np.ndarray

Decodes a SIM800C `CREC` recording (DVI/IMA ADPCM, 4-bit, 8 kHz, headerless) via stdlib `audioop` (`adpcm2lin` + `ratecv`), normalising to float32.

ffmpeg_available() -> bool

`True` if `ffmpeg -version` runs successfully.

Module: gsm_modem.py

Low-level AT-command driver for the USB GSM/GPRS modem — primarily the SIM800C on COM6 (via a CH340 USB-serial bridge). It is the call-signalling layer: place/answer/hang up voice calls, send/receive SMS, report network/SIM status, send DTMF, and manage the flash file system used for turn-based audio. A background reader thread continuously parses the serial stream, dispatching URCs (RING, +CLIP, +CMTI, NO CARRIER…) and synchronising command responses. Depends on `pyserial`. Terminators: `OK ERROR NO CARRIER BUSY NO ANSWER NO DIALTONE`.

@dataclass CmdResult

`cmd`, `final` (final token or `None` on timeout), `lines`, `ok`. `value(prefix)` returns the text after `<prefix>:` in the first matching line.

class GsmModem

Class attribute `fast_baud = 460800` (a faster UART baud for file transfers; `0` disables).

GsmModem.__init__(self, port="COM6", baud=115200, on_incoming_call=None, on_call_ended=None, on_sms=None)

`on_incoming_call(number)` fires on RING+CLIP (number or `None`); `on_call_ended()` fires on NO CARRIER/BUSY/NO ANSWER/NO DIALTONE; `on_sms(index)` fires on +CMTI.

GsmModem.open(self)

Opens the port and initialises the modem. Sends `AT`; retries at `fast_baud` if no response; raises `RuntimeError` if the modem does not respond at either baud (a guard against a mute port). Then `init()` and `_raise_baud()`.

GsmModem.init(self)

`AT`; `ATE0` (echo off); `AT+CMEE=2` (verbose errors); `AT+CMGF=1` (SMS text mode); `AT+CSCS="GSM"`; `AT+CLIP=1` (caller-ID); `AT+CNMI=2,1` (new SMS via +CMTI URC).

GsmModem.dial(self, number) -> bool

Places an outgoing voice call: `ATD<number>;` (the trailing `;` marks it voice). Validates `number` with `is_valid_phone_number` first (a `;` +CR could otherwise inject arbitrary AT commands).

GsmModem.answer / hangup / call_state / send_dtmf

`answer()` → `ATA`; `hangup()` → `ATH`; `call_state()` → parsed `+CLCC` (or `"idle"`); `send_dtmf(digits)` → `AT+VTS=<d>` per digit.

GsmModem.status(self) -> dict

`{signal_rssi (AT+CSQ 0–31, 99=unknown), registered (AT+CREG? status 1/5), sim_inserted (AT+CSMINS?), pin (AT+CPIN?), operator (AT+COPS?), raw}`.

SMS: send_sms / read_sms / delete_sms

`send_sms(number, text)` pauses the reader for raw I/O, writes `AT+CMGS="<number>"`, waits the `>` prompt, writes the body + Ctrl-Z (`0x1A`); sends ESC (`0x1B`) to abort if the prompt never arrives. `read_sms(index)` → `AT+CMGR`; `delete_sms(index)` → `AT+CMGD`.

Flash file system

`fs_size(filename)` → `AT+FSFLSIZE`; `fs_write(filename, data)` → `FSCREATE` + `FSWRITE` (raw bytes, `>` prompt, ESC-abort on failure); `fs_read(filename, chunk=4096)` reads in blocks via `AT+FSREAD=<file>,1,<want>,<offset>` (reads exactly payload+terminator because the bytes "OK" occur naturally inside binary audio), rerouting call-drop URCs; `fs_delete(filename)` → `AT+FSDEL`.

In-call audio (turn-based)

`play_media(filename, path=3, volume=80)` → `AT+CMEDPLAY=1,"<file>",<path>,<volume>` (toward the caller); `stop_media()` → `AT+CMEDPLAY=0`; `record_path(file_id=1)` → `C:\REC\<id>.DVI`; `record_start(file_id=1, channel=2)` deletes any prior file then `AT+CREC=1,<id>,<channel>` (channel 2 = the caller's voice/downlink); `record_stop()` → `AT+CREC=2`.

Source note. The SIM800C's in-call audio routing (`CMEDPLAY` reaching the uplink, `CREC` capturing the downlink) is marked in the source as *not yet validated with a live SIM*; the file system is confirmed. Recordings are DVI/IMA ADPCM (via `audioop`) while TTS playback uses AMR-NB (via ffmpeg) — two different codecs on the two directions.

Module: gsm_call.py

Orchestrates a **turn-based** voice conversation over an already-established SIM800C call. Each turn: the AI speaks (text → Handy TTS → PCM → AMR-NB → `FSWRITE` → `CMEDPLAY`), then listens (`CREC` for a fixed window → `FSREAD` → DVI/ADPCM → PCM 16 kHz → Handy STT → text), then the brain replies. No live audio ⇒ no downlink VAD ⇒ a fixed window (`gsm_record_window_s`) per turn.

GsmCallSession.__init__(self, cfg, modem, brain)

`modem` is an opened `GsmModem` (whose `on_call_ended` should call `session.mark_ended()`); `brain` is the conversational backend.

GsmCallSession.run_outgoing(self, number, greeting="Pronto?")

Dials; on failure logs the modem status and returns. Waits `gsm_answer_wait_s` (the SIM800 lacks a reliable "answered" URC), then runs `_conversation(opening=greeting)`.

GsmCallSession.run_answer(self)

Answers (`ATA`), waits 1 s, runs the conversation with opening "Pronto?".

GsmCallSession._conversation(self, opening)

The main loop, guarded by `call_deadline = now + gsm_max_call_s` and `empty_turns`. It ends on the deadline, on `_ended` (call dropped), or after `gsm_max_empty_turns` consecutive silent turns. Per turn: speak any opening, `_listen(gsm_record_window_s)`, then `_think` / `_speak`. `finally` always hangs up.

_think / _speak / _listen / _stt

`_think(text)` streams the brain and returns the reply. `_speak(text)` synthesises via Handy TTS, encodes to AMR-NB, `fs_write`s to `gsm_tts_file`, and `play_media`s it at `gsm_play_volume` (then sleeps the clip duration). `_listen(window_s)` records channel 2 for a fixed window, reads the `.DVI`, decodes via `dvi_to_pcm`, and runs `_stt`. `_stt(pcm16k)` streams to a `HandySttClient` in ~100 ms blocks and returns the joined text.

Module: phone_client.py

HTTP client acting as the "remote control" for the Android gateway phone, mapping 1:1 to the phone's `ControlService` REST API. This is the control plane for the HFP path (the audio plane being `audio_hfp.py`).

class PhoneError(RuntimeError)

Raised on any communication/action error.

PhoneControlClient.__init__(self, config, timeout=10.0)

Uses `config.base_url()` and sets `Authorization: Bearer <auth_token>` on every request.

PhoneControlClient methods

`health()` → GET `/health`, `True` if `status=="ok"`. `status()` → GET `/status` (call state dict `{state: IDLE|RINGING|OFFHOOK, number, since}`). `call(number)` → POST `/call` `{number}`. `answer()` → POST `/answer`. `hangup()` → POST `/hangup`.

Module: media.py

A small HTTP file server that delivers Hermes-generated media (images/video) to the Android app by URL instead of inline over the WebSocket. The gateway registers a file to obtain a token and tells the app `{type:"media", kind, token, port}`; the app downloads from `http://<host>:<port>/media/<token>`. It also accepts images uploaded by the app for the next utterance.

_to_local_path(src) -> str | None

Resolves a Windows path, `file://` URL, or WSL path (mapped to UNC `\\wsl.localhost\<distro>\...`, trying `Ubuntu`, `Ubuntu-22.04`, `Ubuntu-24.04`) to an existing file.

register_file(src) -> str | None

Stores a resolvable file under a random hex token and returns it.

get_path(token) -> str | None

Thread-safe token → local path lookup.

save_incoming_image(data_b64, mime) -> str

Saves an app-uploaded image (extension from `mime`, default `.jpg`) to a temp directory and returns the path.

_Handler / start_media_server(host, port)

`do_GET` serves only `/media/<token>` paths, supports HTTP Range requests (206 Partial Content, enabling seek/streaming for video), sets permissive CORS, and streams in 64 KiB blocks. `start_media_server` runs a `ThreadingHTTPServer` in a daemon `media-http` thread.

How-to procedures

Place an outgoing call — HFP path: ensure the phone is paired (HFP) with the control app running; `PhoneControlClient(config).call("+39…")`; once `OFFHOOK`, open `HfpAudioIO(in_idx, out_idx, rate, block_ms)` (indices from `find_hfp_devices()`).

Place an outgoing call — GSM path: `GsmModem(gsm_port, gsm_baud, on_call_ended=…).open()`, confirm `status()['registered']`, then `GsmCallSession(cfg, modem, brain).run_outgoing("+39…", greeting="…")`, or CLI `python -m callbridge.gsm_call +39… --brain acp`.

Receive/answer: HFP — poll `PhoneControlClient.status()` for RINGING, then `answer()`, then open `HfpAudioIO`. GSM — register an `on_incoming_call` callback, then `run_answer()` (issues `ATA`).

CHAPTER 17

AI Service Clients

This chapter documents the client modules that connect the suite to its AI backends: the Hermes agent (over ACP or HTTP), Handy speech-to-text and text-to-speech, the MuseTalk avatar/lipsync worker, and the wikibridge knowledge service.

Module: acp_client.py

Connects to the **full Hermes agent** over ACP (Agent Client Protocol), a newline-delimited JSON-RPC 2.0 protocol over the agent process's stdio. Unlike `HermesClient` (limited phone-server over HTTP), this reaches the complete agent with all tools/actions. It launches `hermes acp` (WSL or native), maintains a persistent session (preserving context), and exposes `respond(text) → Iterator[str]` — a drop-in brain. The validated flow is `initialize → session/new → session/prompt`, with reply text arriving as `session/update` notifications of type `agent_message_chunk`. Reverse requests are handled: `session/request_permission` is auto-allowed (autonomy); `fs/*` and `terminal/*` are declined.

AcpAgentClient.__init__(self, config)

Initialises process handle, inbound queue, request-id counter, session id, a prompt lock and a start lock, and a `_last_fail` timestamp for a 30-second start cooldown. No process launched here.

warm_up(self) -> None

Starts the agent ahead of time (best-effort) so the ~90 s startup happens once, not on the first utterance.

is_ready (property) -> bool

`True` only if started and the process is alive. Used by `/diag/acp`.

_start_locked(self)

Launches the process and performs the handshake. **WSL mode** (`acp_use_wsl` true): `wsl.exe -d <distro> -- bash -lc <acp_wsl_command>`, cwd `acp_cwd`. **Native mode** (false): the command parsed from `acp_native_command` via `shlex.split(posix=False)`, cwd `acp_cwd_native`, optional `HERMES_HOME=acp_hermes_home`, launched with `CREATE_NO_WINDOW` (no console pops up under the PTT supervisor). Sends `initialize` then `session/new` (reads `sessionId`), each within `acp_init_timeout_s`.

_ensure_started(self) -> bool

Returns `True` if alive. Applies a 30 s cooldown after a failed start (avoids repeated `acp_init_timeout_s` waits), then tries `_start_locked()` under the start lock.

close(self) / end(self)

`close()` clears state and terminates the process (3 s wait). `end()` is a no-op (the ACP session is persistent, keeping context).

_handle_reverse(self, msg) -> bool

`session/request_permission` → selects an "allow" option (auto-allow) or cancels; `fs/*` / `terminal/*` → JSON-RPC error `-32601`.

respond(self, text, preamble=None) -> Iterator[str]

The primary brain API. Acquires the prompt lock. If `_ensure_started()` fails, yields the speakable fallback "Scusa, l'assistente non è al momento disponibile." and returns. Drains stale messages, sends `session/prompt` with `{sessionId, prompt:[{type:"text", text: (preamble or acp_voice_preamble) + text}]}`, and yields `agent_message_chunk` text until the request id closes or `acp_response_timeout_s`. In `finally` it calls `_cancel_turn(rid)` on barge-in/timeout so aborted-turn chunks are not spoken next turn.

Module: hermes_client.py

HTTP + SSE client for the **limited Hermes phone-server** (the legacy, non-ACP path). Errors are logged but never raised: if Hermes goes down mid-call, the orchestrator keeps handling the call (just without replies). Hermes is an HTTP server at `hermes_base_url` exposing `/incoming-call`, `/outgoing-call`, `/utterance` (SSE), `/call-ended`, `/health`.

class HermesError(RuntimeError)

Exception for communication errors.

HermesClient.__init__(self, config, timeout=10.0, read_timeout=120.0)

`timeout` = connect/request timeout; `read_timeout` = per-chunk read timeout on the SSE stream (a mid-stream stall of this many seconds is treated as end-of-turn).

notify_incoming / notify_outgoing(self, session_id, number, voice, lang) -> bool

POST `/incoming-call` / `/outgoing-call` with `{session_id, number, voice, lang}`. `True` on success, `False` (logged) on request exception.

respond(self, session_id, text, direction) -> Iterator[str | dict]

POST `/utterance` `{session_id, text, direction}` (streaming). Yields **str** text chunks and **dict** metadata (e.g. `{"voice_change": {...}}` → the orchestrator calls `tts.configure`). Terminates on `{"done": true}` or error. Never raises.

call_ended(self, session_id) -> bool

POST `/call-ended`.

class HermesSession

A session bound to one `session_id`, exposing `.respond(text)` (delegates to `client.respond(session_id, text, direction)`) and `.end()` (`call_ended`). This is the object handed to `StreamingSession`.

Module: stt_client.py

WebSocket client for Handy's streaming speech-to-text (`WS /stt/stream`). Sends continuous PCM (float32 mono 16 kHz) and receives transcribed utterances. Constant `STT_RATE = 16000`.

HandySttClient.__init__(self, ws_url, connect_timeout=5.0)

Public queues: `utterances` (transcribed text, consumed by the brain thread) and `speech_events` (speech-start, used for barge-in).

start(self) -> None

Runs a `WebSocketApp` in a daemon `stt-ws` thread and waits up to `connect_timeout` for `ready`; on timeout calls `close()` and raises `TimeoutError`.

send_audio(self, pcm) -> None

Sends one PCM block (float32 mono 16 kHz) as a binary frame; no-op if closed; connection-drop errors logged.

close(self) -> None

Closes the WebSocket. The `_on_message` callback dispatches `ready`, `utterance` (→ `utterances`), `speech_start` (→ `speech_events`), `error`.

Module: tts_client.py

WebSocket client for Handy's streaming text-to-speech (`WS /tts/stream`). Sends reply text in pieces and receives synthesised audio sentence-by-sentence.

@dataclass TtsSegment

`samples` (float32 mono PCM), `sample_rate`, `generation` (default 0 — the TTS turn this belongs to; used to discard stale audio on barge-in).

HandyTtsClient.__init__(self, ws_url, connect_timeout=5.0)

Public `audio` queue of `TtsSegment`, `generation` counter, and `_pending_rate` (default 24000).

configure(self, voice="if_sara", lang="it", speed=1.0) -> None

Sends `{"type":"config", voice, lang, speed}`.

send_text(self, chunk) / flush() / bump_generation() / end() / close()

`send_text` → `{"type":"text","chunk"}`; `flush` → `{"type":"flush"}` (synthesise buffered text); `bump_generation` increments the generation on barge-in (older-generation segments are discarded downstream) and returns it; `end` → `{"type":"end"}`; `close` closes the socket.

The TTS binary streaming protocol. Per sentence the server sends `audio_start` (with `sample_rate`, default

24000), then one binary PCM float32 little-endian mono frame, then `audio_end`. The client pairs the declared rate to the following binary frame and wraps it in a `TtsSegment` tagged with the current generation.

Module: handy_client.py

HTTP client for Handy's control channel (`/models/*`). Loads/unloads STT and TTS models in VRAM and queries voices/languages. Base URL from `config.handy_base_url()`.

class HandyError(RuntimeError)

Exception for control-channel errors.

ensure_models_loaded(config, stt_model=None, tts=True, timeout=180.0) -> bool

Module-level, best-effort. POST `/models/load` with `{stt_model, tts}`; returns `resp.ok` or `False` if Handy refuses. The 180 s timeout is generous because a VRAM load can take seconds.

HandyControlClient methods

`models_available()` (GET `/models/available`); `load_models(stt_model=None, tts=True)` (POST `/models/load`); `unload_models(component="all")` (POST `/models/unload`, `component` ∈ `stt` / `tts` / `all`); `models_status()` (GET `/models/status`).

Module: avatar_client.py

Client for the video avatar worker (MuseTalk) — a separate process in a dedicated venv that shows the chosen persona idling on a virtual webcam and, on request, plays a reply with synchronised lip movement. Communicates with newline-delimited JSON over stdin/stdout. The persona archive lives at `avatar/persone/<name>/manifest.json` (`{name, voice:{engine, voice}, clips:[...]}`); the worker picks a random clip on each activation.

AvatarClient.__init__(self, cfg)

State: process handle, lock, ready/persona-ready/persona-error events, `_active_persona`, `last_error`, `preparing` (for GUI polling), and `progress` (`{phase, done, total, percent, eta_s}`).

Archive methods

`avatar_root()` → `Path(cfg.avatar_dir)`; `personas()` → subdirs with a `manifest.json`; `manifest(persona)` → the parsed manifest; `voice_for(persona)` → the manifest `voice` block (or `{}` on error).

start(self, persona) -> bool

Ensures the worker and loads the persona (**blocking** until ready). The first preparation of a clip can take >10 minutes; there is deliberately no absolute timeout, but a no-progress watchdog (`avatar_prepare_stall_s`, default 300 s; `0` disables) aborts a GPU hang. Returns `False` on worker error/death (detail in `last_error`).

start_async(self, persona) -> bool

Runs `start` in a background `avatar-start` thread and returns immediately, exposing state via `preparing` / `last_error` / `_active_persona` (so the GUI polls `/avatar/state` instead of holding an HTTP request open for minutes).

speak_wav(self, wav_path) / interrupt() / stop()

`speak_wav` sends `{"cmd":"speak","wav":...}` (non-blocking, lip-synced); `interrupt` sends `{"cmd":"interrupt"}` (truncate to idle); `stop` sends `{"cmd":"quit"}` (5 s wait, else kill).

_ensure_worker(self) -> bool

Verifies `musetalk-venv/Scripts/python.exe` and `avatar_worker.py` exist; exports `AVATAR_AUDIO_DEVICE` and `CUDA_VISIBLE_DEVICES` (from `avatar_audio_device` / `avatar_gpu_index`); launches the worker; waits up to 180 s for `ready`.

_reader(self)

Dispatches worker events: `ready`; `progress` (stored in `self.progress`, ~2/s); `persona_ready` (clears progress, sets ready); `error` (sets `last_error` and the error event, unblocking `start()` immediately).

Module: knowledge_client.py

Client for **wikibridge** (default :8775), the separate repo hosting a local LLM Wiki organised by project. Function-based (no client class); all functions take a `Config`. wikibridge has no autostart; `ensure_server()` launches it on-demand.

available(cfg) -> bool

GET `/health` (3 s); `True` if OK.

ensure_server(cfg, timeout_s=20.0) -> bool

Starts wikibridge if down and waits for `/health`. Parses `knowledge_serve_cmd`; returns `False` ("wikibridge non installato") if the executable is missing. Launches with `CREATE_NO_WINDOW` + a hidden `STARTUPINFO` (SW_HIDE) so no window appears (deliberately not `DETACHED_PROCESS`, which would ignore `CREATE_NO_WINDOW` and pop a visible window). Polls `available()` up to `timeout_s`.

list_projects(cfg) -> list[dict]

GET `/projects` (5 s); `[]` on error.

project_preamble(cfg, project_id) -> str

Builds the preamble pointing the brain at the project's wiki (`""` if no project or the wiki dir is missing). Path form depends on runtime: WSL form `/mnt/c/...` when `acp_use_wsl`, else Windows path with forward slashes. This is where per-project knowledge isolation is enforced.

push_meeting(cfg, project_id, *, meeting_id, title, date, source, participants, analysis, transcript_path="") -> str | None

Calls `ensure_server(cfg)` first (starts on-demand if a meeting ends while it is down), then POST `/meetings` with the meeting metadata plus `summary` / `highlights` / `agreements` / `tasks` pulled from `analysis`; returns the resulting wiki path.

Integration concepts (for users)

Hermes: ACP vs HTTP. ACP mode (`AcpAgentClient`, enabled by `acp_enabled`) is the complete tool-using agent over stdio, with a persistent session; startup is slow (~90 s) so call `warm_up()` early. Native launch (`acp_use_wsl=false`) runs `acp_native_command` with optional `HERMES_HOME` and `CREATE_NO_WINDOW`; WSL launch (`true`) runs `acp_wsl_command`. HTTP mode (`HermesClient`) talks to the limited phone-server and streams replies as SSE, resilient to Hermes dying mid-call. Both expose the same `respond(text)` shape.

STT/TTS with Handy (:8720). Control (HTTP `/models/*`), STT stream (`ws://…/stt/stream`, 16 kHz PCM in, JSON `utterance` / `speech_start` out), TTS stream (`ws://…/tts/stream`, `config` / `text` / `flush` / `end` in, `audio_start` +binary+ `audio_end` out).

Voice cloning flow. `HandyTtsClient.configure(voice, lang, speed)` selects the voice (stock or a cloned voice such as `sara_clone`). The avatar persona manifest's `voice` block (`{engine: "chatterbox"|"kokoro"|"piper", voice}`) is read by `AvatarClient.voice_for(persona)`; the `chatterbox` engine is the cloning path.

CHAPTER 18

REST API Reference

The Control Server is a Flask application on **port 8770** (`control_host` / `control_port`), consumed by the Control Center GUI and by agents. All bodies are JSON unless a multipart upload is noted. Server-wide Bearer authentication uses `control_auth_token` (applied at the app layer, not inside these handlers). Endpoints returning `503` (session/copilot) indicate the server started without PTT/copilot enabled.

Blueprint: session (api_session.py)

A programmatic equivalent of the physical PTT hotkeys, operating on the *same* `PushToTalkManager` / `CopilotController` used by the hotkeys.

POST /session/start

Starts a voice session. Body `mode` (default `"half"`) ∈ `half` / `full` / `limited`. **200** `{"status":"started","mode":<detail>}`. **503** PTT unavailable; **409** `session_active`; **503** `start_failed`; **400** other.

POST /session/stop

Stops the current session. **200** `{"status":"stopped"}` or `{"status":"idle"}`. **503** PTT unavailable.

GET /session/state

Returns `{"active","mode","phase"}`. **503** PTT unavailable.

POST /copilot/toggle

Toggles the copilot. **200** `{"active":<bool>}`. **503** copilot unavailable.

GET /copilot/state

200 `{"enabled","active","project_id","save_state"}`. Always 200 (missing copilot ⇒ `enabled:false`).

POST /copilot/project

Binds the copilot to a wiki project. Body `project_id`. **200** `{"project_id": … }`. **503** copilot unavailable.

POST /copilot/save-meeting

Analyses the current copilot session and files it into the project wiki (async). Body `project_id?`, `title?`. **202** `{"started":true,"save_state": … }`. **503/409**.

Blueprint: personas (api_personas.py)

Manages the video-identity archive (`<avatar_dir>/persone/<name>/` with `manifest.json` + mp4 clips) and drives the MuseTalk avatar worker. Names match `^[a-zA-Z0-9_-]+$`; clips match `^[a-zA-Z0-9_.-]+\.mp4$`. A single per-process `AvatarClient` is created lazily under lock.

GET /personas

Lists personas: `{"personas":[{name, voice, clips, clip_count}, …]}` (unreadable manifests skipped).

POST /personas

Creates a persona. Body `name` (req), `voice?` (default `{}`). **201** `{"status":"ok","name"}`. **400** invalid name; **409** exists.

PUT /personas/<name>

Updates the manifest. Body `voice?` (replaces `manifest["voice"]`). **200** `{"status":"ok","manifest"}`. **404**.

DELETE /personas/<name>

Removes a persona. **200**. **404** not found; **409** persona active on the avatar.

POST /personas/<name>/clips

Uploads one mp4 clip (multipart field `clip` or `file`). **201** `{"status":"ok","clips":[…]}`. **404/400**.

DELETE /personas/<name>/clips/<clip>

Deletes a clip (basename only). **200** `{"status":"ok","clips":[…]}`. **404**.

POST /avatar/start

Asynchronously starts/prepares the avatar for a persona. Body `persona` (req). **200** already running / **202** `{"status":"preparing"}`. **404**/**409**.

POST /avatar/interrupt

Truncates the currently spoken line. **200**.

POST /avatar/stop

Stops the worker and clears the active persona. **200**.

GET /avatar/state

`{"running","persona","preparing","progress","error"}` (`running` requires alive + active + not preparing).

POST /avatar/speak

Synthesises text via Handy `/tts/test` and makes the active avatar pronounce it (backend of the Studio panel). Body `text` (req), `voice?` (default persona voice), `lang?` (default `default_lang`). **200** `{"status":"ok","duration_ms","voice"}`. **409** no active avatar; **400** missing text; **502** Handy error.

Blueprint: config (api_config.py)

Reads and mutates the shared `Config`. "Hot" fields are mutated in place; everything is persisted to `config.json`. Fields in `RESTART_REQUIRED` (control host/port/token, gateway_*, all PTT/copilot hotkeys, `copilot_enabled`, `copilot_use_acp`, `hermes_mode`, `hermes_base_url`, `acp_*` WSL fields, `call_transport`, ...) only take effect after a full restart; the GUI shows a "restart needed" banner.

GET /config

`{"config":<as_dict>,"restart_required_fields":[...]}`.

PUT /config

Applies updates. Body `updates` (req, non-empty). Unknown/type-mismatch fields are rejected; unchanged values skipped; the rest applied and (if any) persisted. **200** `{"applied":[...],"requires_restart":[...],"rejected":{field:reason}}`. **400** if `updates` missing/empty or nothing applied due to rejections.

Blueprint: components (api_components.py)

Reports status of and controls `main`, `gateway`, `handy`, `hermes`, `avatar_worker`. States: `running | loading | stopping | blocked | error | stopped | disabled`. `main` stop/restart reply before exiting (via a Timer); the run-ptt.ps1 supervisor relaunches. Constants: `RESTART_EXIT_CODE = 42`; `STOP_SENTINEL = <call-bridge>/ptt.stop` (while present the supervisor will not relaunch).

GET /components/status

`main` `{state:"running",pid,stopping,session}`; `gateway` `{state,port}`; `handy` (from `/health` + `/models/status`); `hermes` (ACP agent readiness or `/health`); `avatar_worker` `{state,persona}`.

POST /components/main/restart

Deletes the stop sentinel, exits with code 42 after 0.5 s (children cleaned up); the supervisor relaunches. **200** `{"status":"restarting"}`.

POST /components/main/stop

Writes the `ptt.stop` sentinel (supervisor stays down), exits with code 0 after 0.5 s. **200** `{"status":"stopping"}`.

POST /components/handy/<action>

`action` ∈ `start` / `stop` / `restart`. stop/restart run `taskkill /F /IM <handy exe>`; start/restart launch the exe detached. **200**. **400** unknown; **501** no `handy_exe_path`; **504** timeout; **502** exe not found.

POST /components/hermes/<action>

`action` ∈ `start` / `stop` / `restart`, from `hermes_stop_cmd` / `hermes_start_cmd` (via `wsl.exe` when `acp_use_wsl`, else the Windows shell). **200**. **400**/**501**/**504**.

POST /components/gateway/restart

Not supported (the gateway lives in main). **409** `restart_main_instead`.

Blueprint: diag (api_diag.py)

Point diagnostic checks powering the guided-test wizard. `_audio_lock` serialises `sounddevice`.

GET /diag/mic

Query `device?` (default `ptt_input_device`), `seconds?` (default 1.0, clamped 0.2–5.0). **200** `{ok,rms,peak,seconds,device_name}`. **502** device error.

POST /diag/echo

Records the PTT mic and plays it back on the PTT speakers. Body `seconds?` (default 3.0, clamped 0.5–10.0). **200** `{ok,rms,seconds}`. **502**.

GET /diag/loopback

Verifies the WASAPI loopback used by the copilot (1 s capture at `copilot_loopback_samplerate`). **200** `{ok,device_name,rms,note}`. **502**.

GET /diag/handy

200 `{ok,base_url,health,models}`. **502** if `/health` unreachable.

GET /diag/hermes

Delegates to the component brain check. **200** `{ok, ... status}` (`ok` = state running).

GET /diag/acp

Checks the WSL/native runtime and ACP warmth. **200** `{ok,mode,runtime,runtime_ok,wsl_ok,agent_ready,note}`. **501** if `acp_enabled` false.

POST /diag/tts-roundtrip

Runs TTS → STT through Handy and compares (no PC audio devices). Body `text?`. **200** `{ok,spoken,heard,match,tts_duration_s,voice}` (`ok` = `match ≥ 0.6`). **502** with `{step:"tts"|"stt",error}`.

Blueprint: analysis (api_analysis.py)

Analyses video/audio from an uploaded file or URL, one background job at a time. States: `downloading` (URL, yt-dlp) → `extracting` (ffmpeg to mono-16k WAV, split into N-second segments) → `transcribing` (each segment via Handy `/transcribe`) → `analyzing` → `done` / `error`. Artifacts in `<analysis_artifacts_dir>/<id>/`: `source.json`, `transcript.json`, `analysis.json`, `summary.md`.

POST /analysis/start

Multipart `file` (+ `title?`) OR JSON `{url (^https?://), title?}`. Creates `<id> = YYYYMMDD-HHMMSS-<6hex>`. **202** `{"id","state":"queued"}`. **501** no ffmpeg; **409** busy; **400** bad input.

GET /analysis/list

`{"analyses": [{id,title,source_type,source,created,duration_s,state,error}, ...]}` (live job's state overlaid).

GET /analysis/<job_id>

`{id, ... source.json, state, progress, error, analysis?}`. **404**/**500**.

GET /analysis/<job_id>/transcript

Returns `transcript.json` (`{segments: [{start_s,end_s,text}]}`). **404**.

DELETE /analysis/<job_id>

200. **400** unsafe id; **404**; **409** running.

Blueprint: meetings (api_meetings.py)

Drives the meeting bot (one call at a time) and manages meeting history; also exposes calendar OAuth and calendar-driven auto-join. Artifacts in `<meetbot_artifacts_dir>/<id>/`.

POST /meetbot/join

Body `url` (req, http), `display_name?`, `title?`, `project_id?`. **202** `{"session_id","state"}`. **400**/**409**.

GET /meetbot/status

`{"state":"idle"}` or the live session status.

POST /meetbot/leave

Leaves and launches background post-call analysis. **200** `{"state":"idle"|"leaving","session_id?"}`.

GET /meetings/list

`{"meetings": [{id,title,created,ended,participants,segments,analysis_state}, ...]}`.

GET /meetings/<mid>

`{id, ... meta.json, analysis?}`. **404**/**500**.

POST /meetings/<mid>/push-wiki

Inserts an analysed meeting into the project wiki. Body `project_id?`. **200** `{"wiki_path"}`. **404/409/400/500/502**.

GET /meetings/<mid>/transcript

Reads `transcript.jsonl`. **200** `{"segments":[ … ]}`. **404**.

DELETE /meetings/<mid>

200. **400/404/409**.

GET /calendar/status

`{"google":{configured,connected,flow_error},"microsoft":{ … }}`.

POST /calendar/<provider>/connect

`provider` ∈ `google` / `microsoft`. **200** the provider's `start_connect()` result. **400** unknown.

POST /calendar/<provider>/disconnect

200 `{"status":"ok"}`. **400** unknown.

GET /calendar/events

Query `days?` (default 7, clamped 1–31). **200** `{"events":[ … ]}`.

POST /calendar/events/<event_id>/autojoin

Body `enabled` (bool). **200** `{"id","autojoin"}`.

Blueprint: gpu (api_gpu.py)

Enumerates NVIDIA GPUs for the GUI's card pickers (e.g. `avatar_gpu_index`), backed by `nvidia-smi` CSV with a 60 s cache. Empty list if `nvidia-smi` is absent.

GET /gpu/list

`{"gpus":[{index,name,vram_mb},…],"source":"nvidia-smi"|"none"}`.

How-to (typical sequences)

Start/stop a voice session: optionally `GET /components/status`; `POST /session/start {mode}`; poll `GET /session/state`; `POST /session/stop`.

Create a persona, clone a voice, run the avatar: `POST /personas`; `POST /personas/<name>/clips` per clip; `POST /avatar/start` → poll `GET /avatar/state`; `POST /avatar/speak`; `POST /avatar/interrupt` / `stop`; `DELETE /personas/<name>` (409 if active).

Restart a component: Handy `POST /components/handy/restart` (needs `handy_exe_path`); Hermes `POST /components/hermes/restart` (needs the cmds); gateway → `409 restart_main_instead`, so `POST /components/main/restart` instead.

Run diagnostics: `GET /diag/mic`, `POST /diag/echo`, `GET /diag/loopback`, `GET /diag/handy` / `hermes` / `acp`, `POST /diag/tts-roundtrip`.

Launch a media analysis: `POST /analysis/start` → poll `GET /analysis/<id>` → read `analysis` + `GET /analysis/<id>/transcript` → `DELETE`.

Meeting bot: (optional) `GET /calendar/status`, `POST /calendar/google/connect`, `GET /calendar/events`, `POST /calendar/events/<id>/autojoin`; then `POST /meetbot/join` → poll `GET /meetbot/status` → `POST /meetbot/leave` → `GET /meetings/<id>` → `POST /meetings/<id>/push-wiki`.

CHAPTER 19

Meeting, Calendar & Analysis Modules

This chapter documents the meeting-bot subsystem (joins Zoom/Teams/Google Meet, transcribes, summarises), the calendar OAuth watcher that auto-joins scheduled meetings, the shared media-analysis engine, and two small text/language utilities. Configuration keys live on the `Config` object (Chapter 5).

Module: meetbot/bot.py — MeetBotSession

Represents a single meeting as a single bot, driving `join → capture → speech-to-text → leave`. It launches a headless-visible Chromium (Playwright), joins through a platform-specific adapter, captures the PC audio loopback, streams it to STT, and writes a diarised transcript. Because Playwright's synchronous API is not thread-safe, all page operations run on a dedicated Playwright thread driven by a command queue. Constant `SPEAKER_POLL_S = 0.4`. States: `joining → waiting_admit → in_call → leaving → ended | error`.

MeetBotSession.__init__(self, cfg, url, display_name=None, artifacts_dir=None)

`url`'s host selects the adapter. `display_name` falls back to `meetbot_display_name` ("Hermes Bot"). `artifacts_dir` defaults to `<call-bridge>/<meetbot_artifacts_dir>`. Sets `session_id` = `YYYYMMDD-HHMMSS-meet`, `dir = root/session_id` (created), a `TranscriptStore` (→ `transcript.jsonl`), a `SpeakerTimeline`, and a Playwright command queue.

start(self) -> None

Starts the `meetbot-playwright` daemon thread; returns immediately (join outcome observed via `state`).

leave(self) -> None

Enqueues `"leave"` (acted on in the waiting-room or in-call loop).

status(self) -> dict

`{session_id, state, url, platform, participants, transcript_tail (last 8), error}`.

_playwright_main(self) — worker thread

Selects the adapter (`error` if none), launches Chromium (`headless=False`, `--use-fake-ui-for-media-stream`, `--autoplay-policy=no-user-gesture-required`) with camera/mic denied (the bot listens through the loopback), navigates and calls `adapter.join`. Waiting-room loop: up to 10 minutes, honouring `leave`, promoting to `in_call` when `adapter.in_call` becomes true. Then `_start_pipeline()`. Service loop: every 0.4 s drains a command, records the active speaker/participants, and concludes the meeting ended if the call has been absent >15 s. On exit: `leave`, close the browser, teardown, `state="ended"`.

_start_pipeline(self)

Starts a `HandySttClient`, a `WindowsLoopbackAudio(copilot_loopback_samplerate, stt)` (48000 default), and a `meetbot-brain` thread.

_brain_loop(self) — brain thread

Coalesces STT utterances into one turn (`ptt_turn_pause_s`, 1.5 s), attributes the speaker via `timeline.dominant(t_from-3.0, t_to)`, and appends the segment.

Module: meetbot/transcribe.py — SpeakerTimeline & TranscriptStore

Produces the diarised transcript. Each utterance is attributed to whichever speaker the adapter observed active in the time window (`"Sconosciuto"` if unknown). Persistence is append-only JSONL so the transcript survives a crash.

class SpeakerTimeline

`__init__(keep_s=600.0)`; `record(speaker)` appends `(now, speaker)` and prunes old events; `dominant(t_from, t_to)` returns the most frequent speaker in the window (or `None`).

class TranscriptStore

`__init__(path)` records a monotonic baseline for relative `start_s`. `note_participants(names)` dedups new names. `add(text, *, speaker=None, spoken_from=None, spoken_to=None)` appends and persists `{ts:"HH:MM:SS",`

`start_s, speaker, text}`. `tail(n=10)` returns the last `n` segments.

Module: meetbot/audio.py — WindowsLoopbackAudio & PulseAudioBot

The audio backend. The meeting browser plays through the PC speakers; the bot captures the loopback of the default output, resamples to 16 kHz, and feeds the STT. **Structural limitation: one meeting at a time, and no other audio may play during capture.** `WindowsLoopbackAudio.__init__(samplerate, stt)`; `start()` runs the `meetbot-capture` daemon (`sc.get_microphone(name, include_loopback=True)`, ~0.1 s blocks, mono, resample to 16 kHz); `stop()` joins it. `PulseAudioBot` is a stub for a future WSL/PulseAudio multi-bot backend (raises `NotImplementedError`).

Module: meetbot/adapters/base.py — adapter interface

Defines the contract every platform adapter implements, with selectors in class constants for quick repair. `@dataclass JoinResult(ok, state, message)`; `@dataclass SpeakerInfo(active, participants)`. `class MeetingAdapter` (`platform="generic"`): `matches(url)`, `join(page, display_name) → JoinResult`, `in_call(page) → bool`, `speakers(page) → SpeakerInfo` (default empty), `leave(page)`.

Module: meetbot/adapters/meet.py — MeetAdapter (Google Meet)

`platform="meet"`; matches `meet.google.com/`. `join` waits for load, fills the guest name (if visible), turns mic/cam off, clicks join ("Partecipa"/"Join now"/"Ask to join"), and waits up to 20 s for the leave button → `in_call`, else `waiting_admit`. `in_call` checks the leave button. `speakers` reads up to 30 `[data-participant-id]` tiles (name from `aria-label`; active speaker best-effort from a `speaking`/`active` class). `leave` clicks the leave button.

Module: meetbot/adapters/teams.py — TeamsAdapter (Microsoft Teams)

`platform="teams"`; matches `teams.microsoft.com/` or `teams.live.com/`. Forces the web client ("Continue on this browser"), enters the guest name (`[data-tid="prejoin-display-name-input"]`), turns mic/cam off when `aria-pressed=true`, clicks join (`[data-tid="prejoin-join-button"]`), waits for the leave button (`[data-tid="hangup-main-btn"]`) → `in_call`, else `waiting_admit`. `speakers` reads the roster (`[data-tid="participantsInCall"]`), no active-speaker detection.

Module: meetbot/adapters/zoom.py — ZoomAdapter (Zoom)

`platform="zoom"`; matches `zoom.us/j/`, `/wc/`, `/s/`. `_to_web_client` rewrites a `/j/<id>` launch URL to `https://app.zoom.us/wc/<id>/join` (carrying `pwd`). `join` navigates to the web client, fills `#input-for-name`, clicks join, waits up to 30 s → `in_call`, else `waiting_admit`. `speakers` reads participant tiles (names only). `leave` clicks leave then the "Leave Meeting" confirmation.

Module: meetbot/adapters/__init__.py — routing

Registers `MeetAdapter` unconditionally; appends `TeamsAdapter`/`ZoomAdapter` guarded by `try/except ImportError`. `adapter_for(url)` returns an instance of the first adapter whose `matches(url)` is true, or `None`.

Module: meetbot/calendar_watch.py — AutojoinStore & CalendarWatcher

Background watcher polling the calendars every `calendar_poll_s` and auto-joining meetings marked for auto-join (persisted in `artifacts/meetings/autojoin.json`). It schedules the join ~1 minute before start and routes it through the same callback the GUI uses (respecting the one-meeting-at-a-time constraint). Constants `JOIN_EARLY_S = 60`, `JOIN_LATE_TOLERANCE_S = 900`.

_parse_dt(value) -> float | None

Parses ISO-8601 to a POSIX timestamp (accepts trailing `Z`; assumes UTC without a timezone).

class AutojoinStore

`get(event_id)` / `set(event_id, enabled)` (persisted JSON; `False` removes the entry).

class CalendarWatcher

`__init__(cfg, google, microsoft, store, join_cb)`; `start()` / `stop()`; `fetch_events(days=7)` merges both providers (a failing provider only logs), sorts by start, annotates `autojoin`, and returns dicts `{id, provider, title, start, end, organizer, meeting_url, autojoin}`. `_loop` joins events whose `autojoin` + `meeting_url` fall in `[start-60 s, start+900 s]`, marking done only on a successful join (failures retry within the 15-minute window).

Module: calendar_oauth.py — GoogleCalendar & MicrosoftCalendar

Read-only calendar access via OAuth. Google uses `InstalledAppFlow` (localhost redirect, opens the browser); Microsoft uses the device-code flow. Token caches in `call-bridge/secrets/` (git-ignored). Scopes: Google `calendar.readonly`, Microsoft `Calendars.Read`. `extract_meeting_url(*texts)` recovers a Meet/Teams/Zoom link from event text.

class GoogleCalendar

`configured()` (both client id+secret set); `connected()` (valid creds loadable); `start_connect()` runs the flow in a `google-oauth` thread (returns `browser_aperto` / `in_corso` /error); `disconnect()` deletes `google_token.json`; `events(days=7)` lists the `primary` calendar (`singleEvents`, `orderBy=startTime`, `maxResults=50`), join URL = `hangoutLink` or extracted, ids prefixed `g:`.

class MicrosoftCalendar

`configured()` (client id set); `_get_app()` builds a `PublicClientApplication` with a serializable cache and authority for `calendar_ms_tenant` (default `common`); `start_connect()` initiates the device flow (returns `{user_code, verification_uri, detail}`), completed in an `ms-oauth` thread; `connected()`; `disconnect()`; `events(days=7)` GETs Graph `/me/calendarView`, join URL = `onlineMeeting.joinUrl` or extracted, ids prefixed `m:`.

Module: meetbot/summary.py — end-of-meeting analysis

Wraps the shared `callbridge.analysis` engine. `analyze_meeting(cfg, meeting_dir, meta)` reads `transcript.jsonl`, runs `analyze_transcript` (passing observed `participants` and, for project-linked meetings, the project wiki as a preamble), writes `analysis.json` + `summary.md`, updates `meta.json` (`analysis_state = "done"|"error"`), and — for project-linked meetings — pushes to the wiki. `push_meeting_to_wiki(cfg, meeting_dir, meta, analysis, *, source="meetbot")` calls `knowledge_client.push_meeting` and returns the wiki path.

Module: analysis.py — shared transcript analysis engine

Given a transcript, uses the ACP agent to produce a global summary, highlights, agreements, and per-participant tasks. Long transcripts use a **map-reduce**: split into ~10k-char chunks on segment boundaries, a partial JSON per chunk, then a reduce pass merges them. A lazy shared brain (`_brain`, ~60–90 s startup) is created once per process.

analyze_transcript(cfg, segments, *, participants=None, progress=None, kind="riunione", preamble=None) -> dict

The main entry point. `segments` are `{text, speaker?, start_s?}`. `participants` (real names for task attribution), `progress(phase, percent)`, `kind` (artifact

kind), `preamble` (e.g. project wiki). Chunks by `max(2000, analysis_chunk_chars)` (default 10000), runs the map + reduce, and returns a normalised `{summary, highlights:[{title, summary}], agreements:[{title, detail}], tasks:[{assignee, task}]}` (missing assignee → "non specificato").

analysis_to_markdown(title, analysis) -> str

Renders the analysis into `summary.md` (Italian headings: Riassunto, Punti salienti, Accordi, Attività). Empty sections omitted.

Module: lang_map.py — phone-prefix → language/voice

Maps an international dialling prefix to a `(language, voice)` pair for Kokoro TTS (inbound calls). `_PREFIX_MAP`: `39→("it","if_sara")`, `1→("en-us","af_heart")`, `44→("en-gb","bf_emma")`, `33→("fr-fr","ff_siwis")`, `34/52/54→("es","ef_dora")`, `55/351→("pt-br","pf_dora")`, `81→("ja","jf_alpha")`, `86→("cmn","zf_xiaoxiao")`, `91→("hi","hf_alpha")`. Prefixes are checked longest-first (3, 2, 1 digits).

voice_lang_for_number(number, default_lang, default_voice) -> tuple[str, str]

Returns the defaults if `number` doesn't start with `+`; else strips `+` and leading zeros and returns the first prefix match, or the defaults.

Module: text_sanitize.py — TTS text sanitization

Sanitises text destined for TTS, removing material that must not be read aloud: reasoning/meta blocks (`<think>…</think>`), tool-calls emitted as text (`<function>…`, `[TOOL_RESULT]…`), markdown, system markers, and emoji. `sanitize_for_tts(text)` cleans a complete text; `TtsTextFilter` does the same in streaming mode.

sanitize_for_tts(text) -> str

Removes complete blocks, stray/system markers, emoji, and text emoticons; strips markdown (code fences, links/images to their label, emphasis, headings, blockquotes, list markers, rules); normalises whitespace. Returns `""` if the whole text is only a re-read system instruction.

class TtsTextFilter

`feed(chunk)` appends and returns complete cleaned sentences (holding back an open block or a partial sentence); `flush()` emits the remainder (discarding open blocks) and clears the buffer.

How-to & config keys

A meeting bot joins with camera/mic denied, transcribes the loopback (`copilot_loopback_samplerate` → 16 kHz), and writes `transcript.jsonl` plus (after analysis) `analysis.json` / `summary.md` / `meta.json` under `<meetbot_artifacts_dir>/<session_id>/`. Keys: `meetbot_enabled`, `meetbot_backend` (`windows`; `wsl` not implemented), `meetbot_display_name`, `meetbot_artifacts_dir`.

Auto-join polls both calendars over a 7-day window; keys `calendar_google_client_id` / `_secret`, `calendar_ms_client_id` / `_tenant`, `calendar_poll_s`. **OAuth setup:** Google — a "Desktop app" OAuth client, then `start_connect()` (browser). Microsoft — an Azure public-client app with `Calendars.Read`, then `start_connect()` (device code).

Analysis keys: `analysis_artifacts_dir`, `analysis_ffmpeg_path`, `analysis_ytdlp_path`, `analysis_segment_s`, `analysis_chunk_chars`.

CHAPTERS 20–21

Avatar Worker, Companion App & Tools

This chapter documents the Android companion app, the avatar/lipsync video worker, and the auxiliary scripts (Kokoro TTS setup and wrappers, the agent control CLI, the agent-side reference server) and the diagnostics folder.

Part A — Android Companion App

The Android app ("callAPIcall Voice", Capacitor + React + TypeScript, Zustand, react-i18next, Tailwind) is a remote control for the PC-side voice assistant. It replicates the desktop Shift+F1/F2 push-to-talk over the WebSocket gateway: hold a button (or lock it continuous) to speak, the assistant transcribes, reasons, replies in text, and speaks back with a Kokoro TTS voice. It can also attach photos and receive generated images/videos.

End-user guide

Connecting to the gateway (over Tailscale). In **Settings** set the **Gateway URL** (e.g. `ws://100.x.y.z:8771`, the Tailscale IP of the PC running Handy + the gateway; shipped default `ws://100.110.144.5:8771`) and the **Token** (must match the backend `gateway_auth_token`; default placeholder `change-me-control-token`). The app is configured for cleartext HTTP/WS (`androidScheme:"http"`, `cleartext:true`) since it reaches the backend over Tailscale/LAN without TLS. The header Wi-Fi icon is green when connected (`ready`).

Choosing the audio output — this changes how you talk. The three output buttons at the bottom select where the voice plays and the interaction model: **Speaker** (viva voce) → **turn-based** (push-to-talk, because the speaker would echo into the mic); **Earpiece** and **Bluetooth** → **full-duplex** (talk and listen at once). Changing the output first cancels any session in progress, then re-routes native audio.

Talking in Speaker mode (hold-to-talk + drag-up lock). The round Talk button is press-and-hold: hold and speak, release to send ("hold" turn). While holding, **drag up ~60 px** to *lock* continuous streaming (the backend ends the turn on a natural pause). The button colour reflects the phase: listening (accent), reasoning (amber), speaking (pulsing).

Talking in Earpiece/Bluetooth mode (tap to call). The button becomes a phone toggle: tap once (green) to open a continuous full-duplex session; tap again (red) to end.

Attaching an image. Tap the header image icon; the photo (JPEG, quality 80, base64) is attached to the *next* message — a toast confirms, then speak to send it.
Receiving media. Generated images/videos appear as chat bubbles; long-press (≈550 ms) or right-click a media bubble to **Save locally** (to the device Documents folder, `callapicall_<timestamp>.jpg/.mp4`). The trash icon clears the chat; the gear icon opens Settings.

App/module: App.tsx

Root component. On mount loads settings, applies the UI language, constructs the singleton `VoiceController`, and routes native audio (`applyOutput()`). `onSave(s)` persists settings and updates the controller; `onChangeOutput(mode)` cancels any session, saves, and re-routes.

Component: ChatView.tsx

The scrolling transcript (user right, assistant left), inline image/video, a streaming caret while a reply is pending, and a long-press "Save locally" sheet. `save()` calls `saveMediaToDevice(url, kind)`.

Component: OutputSelector.tsx

Three buttons — `earpiece` (Ear), `speaker` (Volume2), `bluetooth` (Bluetooth); clicking calls `onChange(mode)`.

Component: SettingsSheet.tsx

Fields: **Gateway URL**; **Token** (password); **Language** (`it` / `en`, live-switches the UI); **Voice** (`im_nicola`, `if_sara`, `af_heart`, `am_michael`, `ff_siwis`, `ef_dora`); **Save**.

Component: TalkButton.tsx

The central control. `DRAG_THRESHOLD = 60` px. Full-duplex branch (earpiece/bluetooth): a Phone/PhoneOff toggle calling `controller.toggleDuplex()`. Speaker branch: `begin/move/end` handlers call `controller.onPressStart()`, `lockContinuous()` past the threshold, and `onPressEnd()`, using pointer capture so the drag keeps tracking.

Library: lib/gateway.ts — WebSocket protocol client

Low-level client. Types: `TurnMode = "hold"|"continuous"`; `MediaMsg` (`kind`, `url?`, `token?`, `port?`, `mime?`); `GatewayHandlers` (callbacks `onTranscript`, `onReply`, `onAudio`, `onMedia`, `onDone`, `onTurnDone`, `onInterrupt`, `onSessionStarted`, `onError`, `onStatus`). `GatewayClient`: `connect()` (opens the socket, sends `auth`, resolves on `ready`, 8 s timeout); `onMessage` (ArrayBuffer → `onAudio`; JSON dispatched by `type`); `startTurn(mode, lang, voice, sampleRate)`; `sendAudio(pcm16)`; `sendImage(dataB64, mime)`; `endTurn()`; `startDuplex` / `stopDuplex`; `close()`.

Library: lib/controller.ts — turn/session orchestration

`VoiceController` ties together the gateway client, mic capture, playback, native routing, image attach, and the chat store. `ensureConnected()` builds a `GatewayClient` and wires handlers into the store. Turn-based (speaker): `onPressStart()` (connect, buffer/stream chunks), `lockContinuous()` (switch to continuous, `startTurn`, flush the buffer), `onPressEnd()` (hold: `startTurn` + flush + `endTurn`; continuous: wait for done then stop). Full-duplex: `toggleDuplex()`, `startDuplex()` / `stopDuplex()`. Media: `mediaUrl(m)` returns `m.url` or `http://<host>:<port ?? 8772>/media/<token>`. `attachImage()` uses Capacitor Camera (quality 80, base64). Capture is sent at 16000 Hz; playback uses the gateway-reported rate (default 24000, Kokoro).

Library: lib/audio.ts — mic capture & TTS playback (Web Audio)

`STT_RATE = 16000`. `floatToPcm16` converts to little-endian Int16 PCM. `MicCapture.start(onChunk)` opens `getUserMedia` with echo-cancellation/noise-suppression, an `AudioContext({sampleRate:16000})`, and a `ScriptProcessorNode(4096)` that emits PCM16 blocks. `AudioPlayer` queues PCM16 segments for gapless playback (`LEAD = 0.12` s), with `stop()` for barge-in.

Library: lib/audioOutput.ts — native audio routing

`AudioOutputMode = "speaker"|"earpiece"|"bluetooth"`. Wraps a native Capacitor plugin `AudioOutput`: `setAudioOutput(mode)` and `clearAudioOutput()` (no-op on web). `isDuplex(mode)` is `true` for earpiece/bluetooth.

Library: lib/settings.ts — persisted settings

`AppSettings` (`gatewayUrl`, `token`, `lang`, `voice`, `output`), persisted via Capacitor `Preferences` under `"callapicall.settings"`. `DEFAULT_SETTINGS`: gateway `ws://100.110.144.5:8771`, token `change-me-control-token`, lang `it`, voice `im_nicola`, output `speaker`. `loadSettings()` merges over defaults; `saveSettings(s)` stores.

Library: lib/save.ts — save media to device

`saveMediaToDevice(url, kind)` converts to base64, picks the extension, and writes `callapicall_<timestamp>.<ext>` to `Directory.Documents`.

Store: store/chat.ts — Zustand chat state

`useChat` holds `{messages, phase, connected}` with actions `setPhase`, `setConnected`, `addUser`, `addMedia`, `startAssistant`, `appendAssistant`, `finishAssistant`, `clear`. `Phase = "idle"|"listening"|"reasoning"|"speaking"`.

capacitor.config.ts

`appId "com.callapicall.voice"`, `appName "callAPIcall Voice"`, `webDir "dist"`, `server: {androidScheme:"http", cleartext:true}`.

Part B — Avatar / Lipsync Video Worker (avatar/avatar_worker.py)

A dedicated Python process implementing the video avatar on **MuseTalk V1.5**, running in `musetalk-venv` with CWD = the MuseTalk repo. The orchestrator launches it via `avatar_client.py` and speaks **JSON lines on stdin**; the worker emits **JSON event lines on stdout**. Video streams frame-by-frame to an OBS virtual camera; audio plays to a configurable sound device.

stdin commands: `{"cmd":"persona","dir":…,"clip":…}` (load a persona: pick a clip, run MuseTalk preparation — cached on disk, first time takes minutes — then idle); `{"cmd":"speak","wav":…}` (lipsync the audio and play it); `{"cmd":"interrupt"}` (barge-in, handled directly by the stdin reader so it works mid-utterance); `{"cmd":"quit"}`. **stdout events:** `ready`, `persona_ready` (`clip`, `frames`), `speaking`, `progress`, `idle`, `interrupted`, `error`. Constants: `FPS = 25`, `EXTRA_MARGIN = 10`, `PARSING_MODE = "jaw"`, `BATCH_SIZE = 12`.

Startup gotchas. Monkeypatches `torch.load` to `weights_only=False` (trusted legacy MuseTalk checkpoints); inserts `MuseTalk/` on `sys.path` and chdirs into it; imports DWPose/scipy **eagerly before** transformers (importing them lazily inside `prepare_clip` deadlocks the Windows DLL loader).

class PrepProgress

Emits throttled (≤2/s) `progress` events and keeps per-phase speed stats in `cache/prepare-stats.json` (EMA). Phases: `extract`, `landmark`, `latents`, `masks` (masks = 2×n for the forward+reverse loop). ETA blends the live speed of the current phase with historical stats for future phases.

class VideoSink

Tries `pyvirtualcam.Camera(w, h, fps=FPS)` (OBS virtual camera); falls back to an OpenCV preview window. `send(frame_bgr)` outputs (BGR→RGB for the cam).

class Engine — MuseTalk model host and pipeline

Loads models (`unet.pth`, `sd-vae`, `musetalk.json`) at half precision, plus Whisper and `FaceParsing`. `prepare_clip(clip_path, cache_dir)` is the prepare/cache pipeline: cache hit (reload) or extract frames → landmarks (chunks of 25) → VAE latents (crop+resize 256×256) → forward+reverse loop build → blending masks; persists `coords.pkl`, `mask_boxes.pkl`, `latents.pt` and removes the raw frames. `lipsync_stream(mat, wav_path, start_idx)` yields lipsynced frames one at a time in GPU batches of 12 (Whisper audio features + UNet + VAE decode + blend).

play_wav_async(wav_path) / main()

`play_wav_async` plays the wav to `AVATAR_AUDIO_DEVICE` (env; else default). `main()` is the idle/speak loop: builds the `Engine`, emits `ready`, and a `reader()` thread reads stdin (`interrupt` sets an event directly; other commands queue; `quit` breaks). Persona command: reads `manifest.json`, picks a clip at random, `prepare_clip`, (re)builds the sink at the clip resolution, emits `persona_ready`. Speak command: a producer thread runs `lipsync_stream` into a bounded queue, a prebuffer collects `BATCH_SIZE` frames (so audio+video start

together), then `play_wav_async` starts and a consumer renders at 25 fps anchored to the audio clock (discarding late frames, holding the last on a gap). Interrupt stops audio and drains. Idle: cycles the prepared loop at 25 fps.

Persona manifest. A directory with `manifest.json` containing a `"clips"` list; the worker caches prepared material under `cache/<persona>__<clip-stem>/`. Env `AVATAR_AUDIO_DEVICE` sets the output; weights live under `./models/musetalkV15/` and `./models/whisper/`.

Part C — Auxiliary Scripts

kokoro_tts.py — Hermes command-TTS wrapper

A standalone CLI (`python kokoro_tts.py <input_text_path> <output_audio_path>`) that synthesises speech through Handy's Kokoro TTS WebSocket. `WS_URL = ws://127.0.0.1:8720/tts/stream`, `VOICE = if_sara`, `LANG = it`, `SPEED = 1.0`. Sends `config` / `text` / `flush` / `end`, waits up to 8 s for `ready` and 20 s for done, and writes a mono 16-bit **24000 Hz** WAV.

setup/setup_kokoro.py — Kokoro TTS installer/downloader

Installs the Python deps (`kokoro-onnx` , `numpy`) into the Python Handy invokes, and downloads the chosen Kokoro model + voices into Handy's `resources/models` . `MODEL_VARIANTS` : `full` (~310 MB, f32), `fp16` (~169 MB), `int8` (~88 MB, default). `default_models_dir()` prefers the per-user NSIS install, then the MSI, then the dev repo. CLI: `--check` , `--install-deps` , `--download` , `--all` , `--variant {full,fp16,int8}` (default `int8`), `--python` , `--models-dir` .

skill/callctl.py — telephony control CLI (for agents)

A stdlib-only CLI letting an agent place calls, manage models, and read status via the control API. Env `CALLBRIDGE_CONTROL_URL` (default `http://127.0.0.1:8770`), `CALLBRIDGE_CONTROL_TOKEN` (default `change-me-control-token`); Bearer auth, 70 s timeout; exit 0 on 2xx. Subcommands: `status` , `models` , `prepare [--stt-model M]` , `call <number> [--voice V] [--lang L] [--stt-model M]` , `hangup` , `unload [--component stt|tts|all]` .

skill/hermes_server.py — agent-side reference server

The reference implementation (and runnable echo stub) of the agent-side server every LLM runtime must expose to become "the brain" for callAPIcall phone calls. Only dependency: Flask. Hook point `RESPOND_FN = stub_respond` (redefine with a real streaming `respond(session_id, text, direction) → Iterator[str]` ; token-by-token streaming recommended). Endpoints: `GET /health` ; `POST /incoming-call` / `/outgoing-call` (store the session, `{"status":"ready"}`); `POST /utterance` (SSE stream of `data:{"chunk":…}` then `data:{"done":true}`); `POST /call-ended` ; `GET /sessions` (debug). `main()` : `--host` (default `127.0.0.1`), `--port` (default **8780**).

Diagnostics & Probe Utilities (call-bridge/diag/)

The `diag/` folder holds one-shot, throwaway diagnostic scripts from modem/audio debugging. They are **not** part of the `callbridge` package and no module imports them; run them from the `call-bridge/` root as `python diag/<script>.py` . Grouped by purpose:

- **Modem AT probes & flash file system** (`probe_a7670_fs.py` , `probe_a7670_cfs.py` , `probe_ctts.py` , `probe_crec.py` , `probe_crec2.py` , `probe_rec_file.py` , `probe_fsread.py` , `probe_dvi.py` , `probe_dvi2.py`): raw serial AT probes against the SIM800C (COM6) and A7670E (COM9) —

recording (`CREC`), the modem file system, on-modem TTS (`CTTS`), and `.DVI` capture.

- **Baud / bring-up / connectivity** (`test_baud.py`, `test_fast_open.py`, `validate_sim800c.py`).
- **Incoming/outgoing call diagnostics** (`incoming_diag.py` … `incoming_diag4.py`, `outgoing_diag.py`, `multiturn_call.py`): live SIM call-chain tests.
- **Downlink / call-chain / full-pipeline tests** (`test_downlink_chain.py`, `test_voice_io.py`, `test_loopback.py`, `test_copilot_chain.py`): end-to-end path checks. `test_copilot_chain.py` verifies the Shift+F4 loop headlessly (Handy TTS → speakers → soundcard loopback → 16 kHz → Handy STT) and asserts the transcript matches.
- **.DVI audio-decode experiments** (`decode_attempts.py`, `decode_hunt.py`, `confirm_alaw.py`, `inspect_dvi.py`, `analyze_voice.py`, `spectro.py`, `ima_stt.py`): attempts to decode the `.DVI` format from `AT+CREC` (per the folder README this decode is not yet solved).
- **Agent / model warm-up & call helpers** (`warm_hermes.py`, `generate_pitch.py`, `cleanup_modem.py`, `recover_modem.py`): `warm_hermes.py` does a real ACP call to preload the model; `cleanup_modem.py` / `recover_modem.py` reset the modem between tests.
- **ACP/tool capture** (`capture_acp.py`, `capture_acp2.py`, `capture_tool.py`, `capture_tool2.py`): capture/inspect the ACP agent's tool-call traffic.

APPENDICES

Reference Appendices

Quick-lookup indexes: configuration keys, ports and endpoints, a glossary, and a troubleshooting matrix.

APPENDIX A

Configuration Key Index

Every configuration key in alphabetical order with its shipped default, for quick lookup. Chapter 5 groups the same keys by subsystem and explains each in full.

Key	Default
acp_cwd	"/home/atainі"
acp_cwd_native	"C:\Users\atain"
acp_enabled	true
acp_hermes_home	%LOCALAPPDATA%\hermes
acp_init_timeout_s	240.0
acp_native_command	hermes.exe acp --accept-hooks
acp_response_timeout_s	300.0
acp_use_wsl	false
acp_voice_preamble	(short)
acp_wsl_command	(login-shell hermes acp)
acp_wsl_distro	"Ubuntu"
analysis_artifacts_dir	"artifacts/analyses"
analysis_chunk_chars	10000
analysis_ffmpeg_path	"ffmpeg"
analysis_segment_s	300
analysis_ytdlp_path	"yt-dlp"
auth_token	"change-me-shared-token"
avatar_audio_device	null
avatar_dir	"C:\Projects\callAPIcall\avatar"
avatar_enabled	false
avatar_gpu_index	null
avatar_persona	""
avatar_prepare_stall_s	300.0
avatar_prepare_timeout_s	600.0 (deprecated)
barge_in_enabled	true
barge_in_energy_threshold	0.02
block_ms	100
calendar_google_client_id	""
calendar_google_client_secret	""
calendar_ms_client_id	""
calendar_ms_tenant	"common"
calendar_poll_s	300
call_transport	"auto"
control_auth_token	"change-me-control-token"
control_host	"127.0.0.1"
control_port	8770
copilot_audio_reply	false

Key	Default
copilot_enabled	true
copilot_font_size	22
copilot_hide_from_capture	true
copilot_lang	"it"
copilot_loopback_samplerate	48000
copilot_num_responses	3
copilot_preamble	(long)
copilot_scroll_speed	1.0
copilot_tts_output_device	null
copilot_use_acp	true
default_lang	"it"
default_stt_model	null
default_voice	"if_sara"
gateway_auth_token	"change-me-control-token"
gateway_enabled	true
gateway_host	"0.0.0.0"
gateway_media_port	8772
gateway_port	8771
gsm_answer_wait_s	6.0
gsm_auto_answer	true
gsm_baud	115200
gsm_max_call_s	600.0
gsm_max_empty_turns	3
gsm_play_volume	80
gsm_port	"COM6"
gsm_record_id	1
gsm_record_window_s	8.0
gsm_tts_file	"C:\tts.amr"
handy_exe_path	""
handy_host	"127.0.0.1"
handy_port	8720
hermes_base_url	"http://127.0.0.1:8780"
hermes_mode	"stub"
hermes_start_cmd	""
hermes_stop_cmd	""
hfp_input_device	null
hfp_output_device	null

Key	Default
knowledge_api_token	""
knowledge_api_url	"http://127.0.0.1:8775"
knowledge_root	"C:\Projects\wikibridge\data"
knowledge_serve_cmd	wikibridge.exe serve
meetbot_artifacts_dir	"artifacts/meetings"
meetbot_backend	"windows"
meetbot_display_name	"Hermes Bot"
meetbot_enabled	true
phone_host	"192.168.1.50"
phone_port	8080
poll_status_interval_s	1.0
ptt_input_device	null
ptt_listen_warmup_s	0.8
ptt_output_device	null
ptt_post_tts_pause_s	1.0
ptt_turn_pause_s	1.5
push_to_talk_enabled	true
push_to_talk_hotkey	"shift+f1"
push_to_talk_hotkey_copilot	"shift+f4"
push_to_talk_hotkey_fullduplex	"shift+f2"
push_to_talk_hotkey_limited	"shift+f3"
sample_rate	16000
tts_speed	1.2

APPENDIX B

Ports & Endpoints

A one-page map of every network surface the system exposes.

Port	Process	Protocol	Purpose
`8720`	Handy	HTTP / WS	STT (`/stt/stream`), TTS (`/tts/stream`), model & voice management.
`8770`	call-bridge control server	HTTP (Flask)	GUI & agent REST API: sessions, personas, config, components, diag, calls, meetings, analysis.
`8771`	call-bridge voice gateway	WebSocket	Remote voice for the Android app.
`8772`	call-bridge media server	HTTP	Generated media download for the app.
`8775`	Wikibridge	HTTP	Optional per-project knowledge service.
`8780`	Hermes HTTP phone server	HTTP/SSE	Legacy/deprecated agent transport (superseded by ACP).
stdio	Hermes agent	JSON-RPC (ACP)	Native agent conversation.
serial `COM6` / `COM9`	GSM/4G modem	AT commands	Cellular calls & SMS.

APPENDIX C

Glossary

ACP
Agent Control Protocol — JSON-RPC over stdio used to talk to the native Hermes agent.

Barge-in
Interrupting the agent's spoken reply by talking over it (full-duplex only).

Brain
A configured agent instance bound to a hotkey or channel (full, limited, copilot).

HFP
Hands-Free Profile — the Bluetooth profile carrying call audio to/from the phone.

Half/Full duplex
Whether the microphone is muted while the agent speaks (half) or always live (full).

Handy
The Rust/Tauri STT+TTS engine on `:8720`.

Loopback
Capture of the PC's own output audio (WASAPI) — the other party's voice in a call.

MuseTalk
The GPU lip-sync model powering the avatar worker.

Persona
An avatar identity: a folder with a manifest (voice + video clip).

PTT
Push-to-talk — the hotkey-driven voice conversation subsystem.

Split architecture
The supported deployment: headless gateway service + interactive PTT process.

Transport
The physical path a phone call takes: USB GSM modem or Android over HFP.

APPENDIX D

Troubleshooting

Common symptoms and where to look. Most conditions are also surfaced by the Control Center's *Components* dashboard and *Guided Tests*.

Symptom	Likely cause	What to do
Sidebar dot is red / "Disconnected"	The call-bridge control server (:8770) is down.	Open *Components* → Bridge card → **Start** (clears the `ptt.stop` sentinel and launches the supervisor). Confirm with `python main.py ping`.
Pressing Shift+F1 does nothing	PTT disabled, hotkey owned by another app, or the PTT process is not in the user session.	Check `push_to_talk_enabled=true`; ensure the split PTT task is running (`main.py all --no-gateway`); a headless-service-only deployment has no hotkeys.
Session starts then immediately stops (`start_failed`)	Handy models not loaded, or the audio device is unavailable.	Open *Handy* → **Load**; verify the microphone with *Guided Tests* → `/diag/mic`; set `ptt_input_device` / `ptt_output_device` if the default device is wrong.
Agent's first reply takes ~60–90 s	The ACP agent is cold-starting (loading tools/MCP).	Expected once per process; the PTT process warms the agent in the background at launch. Keep the process resident.
Agent speaks its tool calls aloud	An over-heavy `acp_voice_preamble` disrupts tool-calling.	Keep the voice preamble minimal (the shipped default is minimal); the TTS sanitizer already strips markup.
Echo / the agent interrupts itself	Speaker audio re-entering the mic in half-duplex, or the barge-in gate too low.	Use headphones (F2) for full-duplex, or raise `barge_in_energy_threshold`; tune `ptt_post_tts_pause_s` / `ptt_listen_warmup_s`.
No transport for a phone call	GSM modem not registered, or the Android control app unreachable.	*Calls* → **Detect**; for GSM confirm SIM registration (`python -m callbridge.gsm_modem status`); for Android verify the phone app and `auth_token`.
GSM call audio is silent	The SIM800C's in-call file audio routing is not yet validated on the hardware.	Known limitation (see Chapter 16); the flash file system works but in-call `CMEDPLAY` / `CREC` routing awaits SIM validation.
Handy build fails (whisper-rs-sys)	LLVM 22 breaks bindgen.	Install `libclang==18.1.1` via pip and point `LIBCLANG_PATH` at it (see Chapter 4).
Avatar preparation seems stuck	First-time clip preparation is genuinely slow (minutes), or the GPU hung.	Watch the progress bar/ETA; there is no hard timeout, but a no-progress watchdog (`avatar_prepare_stall_s`, 300 s) aborts a true hang.
Virtual camera not found in Meet/Zoom	The "OBS Virtual Camera" DirectShow driver is not installed.	Install the standalone driver from `drivers/obs-virtualcam/` (no OBS Studio required).
Copilot window is blank	Loopback capture unavailable, or nothing is playing on the PC.	*Guided Tests* → loopback (`/diag/loopback`); ensure the other party's audio plays through the default output; install `soundcard` if prompted.
Meeting bot cannot join	Playwright/Chromium missing, or the bot is stuck in the waiting room.	`pip install playwright; playwright install chromium`; an organiser must admit the guest within 10 minutes; check the `join-error.png` / `not-admitted.png` screenshots in the meeting artifacts folder.
Calendar events do not appear	OAuth not completed, or the token expired.	*Meetings* → connect Google/Outlook via the OAuth wizard; Google apps left in "testing" expire every 7 days — publish the consent screen.
Gateway unreachable from the phone app	Wrong Tailscale IP/port or token mismatch.	Set the app's **Gateway URL** to `ws://<PC-tailscale-ip>:8771` and the **Token** to match `gateway_auth_token`.

FIRST LINE OF DEFENCE

The five *Guided Tests* in the Control Center exercise the whole pipeline step by step and pinpoint the failing stage faster than any manual check.

callAPIcall — User Manual · Edition 1.0 · Generated from source · © the project author. Reproduction for internal operator training is permitted.